Revista Brasileira de Informática na Educação, 20XX, VV:1
ISSN: 1414-5685 (print), 2317-6121 (online) • doi: 10.5753/rbie.yyyy.id
This work is licensed under a Creative Commons Attribution 4.0 International License.

Artigo de Pesquisa / Research Paper / Artículo de investigación

Kata: Ferramenta de Verificação Executável para Apoiar o Ensino de Desenvolvimento de Software com Agentes de Inteligência Artificial

Kata: Executable Verification Tool to Support the Teaching of Software Development with Artificial Intelligence Agents

Kata: Herramienta de Verificación Ejecutable para Apoyar la Enseñanza de Desarrollo de Software con Agentes de Inteligencia Artificial

Omitido para revisão

Omitido para revisão.

Resumo. *Introdução.* Agentes de inteligência artificial (IA) escrevem código mais rápido do que qualquer pessoa o revisa, e a supervisão acaba restrita à suíte de testes, que um agente pode satisfazer sem resolver o problema. O mesmo risco existe em disciplinas de programação em que estudantes usam esses agentes. *Métodos.* Este artigo propõe uma ferramenta e um protocolo de estudo. Apresentamos o kata, uma ferramenta de código aberto que conduz uma mudança em cinco domínios (programação, infraestrutura, análise de dados, pesquisa e documentação) por um ciclo de nove fases e a confronta com o repositório por meio de um verificador adversarial (JUDGE), que reexecuta as verificações declaradas e procura oito categorias de fraude. Fundamentamos o ciclo em teoria de ensino, propomos dois casos de uso para um primeiro teste e seis extensões, descrevemos os limites conhecidos do verificador e propomos um protocolo de estudo com estudantes. As regras específicas dos quatro adaptadores de domínio são instruções ao agente e não foram avaliadas. Os dados da verificação técnica, relatados à parte, não são reproduzidos. *Resultados.* A proposta reúne a correspondência entre as fases e os processos cognitivos de Bloom, dois casos de uso para o primeiro teste e seis extensões, cinco desvios do verificador e um protocolo experimental randomizado em grupos paralelos, que prevê cerca de 142 estudantes por grupo. Oito execuções reais, nos cinco domínios, ilustram o ciclo e mostram que o kata executa o que o projeto declara e confere a presença do raciocínio, mas não sua qualidade. *Discussão.* O artigo não mede aprendizagem nem custo de tempo com estudantes. A ferramenta só verifica a presença e a coerência dos registros, e seus desvios conhecidos impedem usá-la como prova única de integridade. O protocolo é o passo seguinte.

Palavras-chave: Educação em Computação; Agentes de Inteligência Artificial; Verificação de Software; Retorno Formativo; Integridade Acadêmica; Engenharia de Software; Ensino de Software.

Abstract. *Introduction.* Artificial intelligence (AI) agents write code faster than anyone can review it, so oversight collapses onto the test suite, which an agent can satisfy without solving the problem. The same risk exists in programming courses where students use these agents. *Methods.* This article proposes a tool and a study protocol. We present kata, an open-source tool that guides a change in five domains (programming, infrastructure, data analysis, research and documentation) through a nine-phase cycle and confronts it with the repository through an adversarial verifier (JUDGE) that re-runs the declared checks and hunts eight categories of fraud. We ground the cycle in teaching theory, propose two use cases for a first trial and six extensions, describe the verifier's known limits, and propose a study protocol with students. The specific rules of the four domain adapters are instructions to the agent and were not evaluated. The data of the technical verification, reported elsewhere, are not reproduced. *Results.* The proposal comprises the mapping between phases and Bloom's cognitive processes, two use cases for the first trial and six extensions, five verifier bypasses, and a parallel-group randomized experimental protocol that plans about 142 students per group. Eight real runs, in the five domains, illustrate the cycle and show that kata runs what the project declares and checks the presence of reasoning, but not its quality. *Discussion.* The article measures neither learning nor time cost with students. The tool only checks the presence and coherence of records, and its known bypasses prevent its use as sole proof of integrity. The protocol is the next step.

Keywords: Computing Education; Artificial Intelligence Agents; Software Verification; Formative Feedback; Academic Integrity; Software Engineering; Software Education.

Resumen. *Introducción.* Los agentes de inteligencia artificial (IA) escriben código más rápido de lo que cualquier persona lo revisa, y la supervisión queda limitada a la batería de pruebas, que un agente puede satisfacer sin resolver el problema. El mismo riesgo existe en asignaturas de programación donde el estudiantado usa esos agentes. *Métodos.* Este artículo propone una herramienta y un protocolo de estudio. Presentamos kata, una herramienta de código abierto que conduce un cambio en cinco dominios (programación, infraestructura, análisis de datos, investigación y documentación) por un ciclo de nueve fases y lo confronta con el repositorio mediante un verificador adversarial (JUDGE), que vuelve a ejecutar las verificaciones declaradas y busca ocho categorías de fraude. Fundamentamos el ciclo en teoría de la enseñanza, proponemos dos casos de uso para una primera prueba y seis extensiones, describimos los límites conocidos del verificador y proponemos un protocolo de estudio con estudiantes. Las reglas específicas de los cuatro adaptadores de dominio son instrucciones al agente y no fueron evaluadas. Los datos de la verificación técnica, reportados aparte, no se reproducen. *Resultados.* La propuesta reúne la correspondencia entre las fases y los procesos cognitivos de Bloom, dos

casos de uso para la primera prueba y seis extensiones, cinco desvíos del verificador y un protocolo experimental aleatorizado de grupos paralelos, que prevé unos 142 estudiantes por grupo. Ocho ejecuciones reales, en los cinco dominios, ilustran el ciclo y muestran que kata ejecuta lo que el proyecto declara y comprueba la presencia del razonamiento, pero no su calidad. *Discusión.* El artículo no mide el aprendizaje ni el costo de tiempo con estudiantes. La herramienta solo comprueba la presencia y la coherencia de los registros, y sus desvíos conocidos impiden usarla como prueba única de integridad. El protocolo es el paso siguiente.

Palabras clave: Educación en Computación; Agentes de Inteligencia Artificial; Verificación de Software; Retroalimentación Formativa; Integridad Académica; Ingeniería de Software; Enseñanza de Software.

Received: DD Month YYYY • **Accepted:** DD Month YYYY • **Published:** DD Month YYYY

1 Introdução

Agentes de programação baseados em IA generativa já resolvem problemas de programação que se pedem a estudantes. Ao comparar o desempenho de um modelo de código em questões reais de provas de programação introdutória com o de estudantes que as fizeram, Finnie-Ansley *et al*. [2022] observaram que o modelo superava a maioria deles. Hoje esses agentes não apenas sugerem trechos: editam arquivos, executam comandos e declaram a tarefa concluída. A Educação em Computação [Bispo Jr. *et al*., 2020] precisa decidir o que ensinar e como avaliar diante de modelos que geram código a partir de descrições em linguagem natural, tema de um relatório internacional e de uma síntese recente [Prather *et al*., 2023a; Denny *et al*., 2024a]. A evidência inicial indica que o efeito da IA generativa sobre iniciantes é desigual. Em um estudo de laboratório, alguns estudantes que não tinham dificuldade aceleraram o trabalho, e nos que tinham, as dificuldades metacognitivas persistiram e a ferramenta pôde agravá-las [Prather *et al*., 2024].

O ensino de desenvolvimento de software, porém, não se limita a escrever programas. Uma meta-análise sobre o alinhamento entre a formação e a indústria aponta que as maiores lacunas incluem gestão de configuração, processo de software e testes [Garousi *et al*., 2019]. Na nossa leitura, e não na da fonte, essas lacunas se materializam em arquivos de infraestrutura, fluxos de integração, notebooks de dados e documentos, e não só em código de aplicação. Os agentes de código atuam sobre todos esses artefatos. Editam um *Dockerfile*, um *notebook*, um relatório ou uma especificação com a mesma facilidade com que editam um programa, e cada artefato tem a sua forma de ser verificado e a sua forma de ser fraudado. O kata foi desenhado para esse conjunto. Ele tem um ciclo comum de nove fases e cinco domínios (programação, infraestrutura, análise de dados, pesquisa e documentação), que variam o que se deve abrir, observar e vigiar antes de declarar uma tarefa concluída (Seção 4).

O problema que este artigo trata é anterior à sala de aula e a atravessa. Quando um agente diz "terminei", nada executa as verificações que ele afirma ter feito. Ele pode alegar que o código passa na análise estática e nos testes sem que nenhum comando tenha rodado. Pode também ter rodado tudo, mas depois de trocar uma asserção por `pass`, de modo que o teste passa sem verificar coisa alguma. O *SpecBench*, um *preprint* ainda sem revisão por pares, mede esse fenômeno, conhecido como *reward hacking*. Ele relata que a distância entre os testes visíveis e os testes ocultos cresce 28 pontos percentuais a cada aumento de dez vezes no tamanho do código [Zhao *et al*., 2026]. É a lei de Goodhart, cujas variantes um *preprint* organiza [Manheim & Garrabrant, 2019], aplicada ao código: a suíte de testes vira a meta e deixa de ser a medida. Em uma disciplina de programação, o corretor automático de exercícios [Keuning *et al*., 2018] está exposto ao mesmo risco, seja quem o contorna um agente ou uma pessoa.

Duas propostas respondem com disciplina. A primeira é o conjunto das quatro diretrizes que um repositório de terceiros extraiu de observações de Karpathy sobre erros comuns de modelos ao programar: pensar antes de codificar, simplicidade primeiro, mudanças cirúrgicas e execução guiada por objetivos [forrestchang, 2026]. A segunda é o Método Fable, que manda classificar a tarefa antes de agir, exigir evidências e verificar de forma adversarial [Sahir619, 2026]. O kata chama de "Ciclo de Desenvolvimento de Karpathy" o ciclo que organiza essas diretrizes em fases, e o nome foi dado pelo projeto kata, e não por Karpathy. Ambas as propostas são, porém, texto que instrui, e um modelo pode seguir a instrução, ignorá-la ou segui-la pela metade sem que ninguém perceba a diferença. Modelos de linguagem também não se autocorrigem de forma confiável sem retorno externo [Huang *et al*., 2024].

Este artigo propõe o **kata**, uma ferramenta de código aberto que transforma essa disciplina em um ciclo de nove fases, como apoio ao ensino de desenvolvimento de software com agentes de IA. A parte objetiva do ciclo é código Python testado, e não instrução ao modelo. É preciso dizer o que o artigo é. É uma **proposta de ferramenta e de protocolo de estudo**: descreve a ferramenta e seus limites conhecidos, fundamenta sua adoção em teoria de ensino, propõe casos de uso e um estudo com estudantes. Não avalia efeito sobre a aprendizagem, e nada nele autoriza afirmar que o kata ensina melhor. A verificação técnica da ferramenta, com agentes de IA como sujeitos, é relatada em um manuscrito nosso, separado deste, que não é disponibilizado nesta submissão, e seus dados não são reproduzidos aqui. As questões que guiam a proposta são:

- **QP1.** Que fundamentação pedagógica sustenta o uso do kata no ensino de desenvolvimento de software com agentes de IA, e que evidência cada fase deixa para o retorno formativo?
- **QP2.** Em que situações de ensino de software o kata poderia ser usado, com que fluxo e com que ressalvas, dados os limites conhecidos do verificador?
- **QP3.** Que estudo com estudantes testaria o efeito da ferramenta sobre a aprendizagem?

As contribuições são cinco: (i) o kata e a descrição de seus algoritmos objetivos; (ii) uma fundamentação pedagógica do ciclo, apresentada como hipótese de projeto; (iii) dois casos de uso para um primeiro teste em sala e seis extensões, cada um com fluxo, evidência produzida e ressalva; (iv) os limites conhecidos do verificador, reproduzidos e documentados; e (v) um protocolo de estudo com estudantes, ainda não executado. **O artigo não apresenta dados de estudantes.** As três questões são respondidas por análise conceitual, e os casos de uso e o protocolo são hipóteses a testar.

A Seção 2 posiciona o trabalho. A Seção 3 descreve o kata e seus limites conhecidos, e a Seção 4 destaca seus cinco domínios. A Seção 5 apresenta a fundamentação pedagógica e a Seção 6 os casos de uso. A Seção 7 propõe o estudo com estudantes, a Seção 8 discute as implicações, a Seção 9 reúne as limitações e a Seção 10 conclui.

2 Trabalhos Relacionados

Critérios de seleção. Este artigo não é uma revisão sistemática da literatura inteira. Para a produção nacional, fizemos uma busca sistemática curta em 3 de outubro de 2026 no SOL (SBIE, WEI e EDUCOMP) e na RBIE, com oito *strings* em português para 2022 a 2026. Dos 126 registros obtidos (75 distintos, 52 no período), incluímos 27, por triagem de título e resumo feita por uma única pessoa, com apoio de um assistente de IA e sem concordância entre avaliadores. O protocolo completo está no Apêndice B, e os registros, a triagem e o código em `busca/`. Para a literatura internacional, reunimos obras por conhecimento da área e por citação, sem busca sistemática, e priorizamos obras revisadas por pares, marcando as que não são (*preprints* e repositórios) no texto e na lista de referências.

IA generativa na Educação em Computação. Os primeiros trabalhos mediram a capacidade dos modelos em tarefas de disciplinas [Finnie-Ansley *et al*., 2022] e discutiram oportunidades e desafios da geração automática de código para o ensino [Becker *et al*., 2023]. Um relatório de grupo de trabalho internacional combinou revisão de 71 artigos, um levantamento com estudantes e docentes de 20 países, entrevistas com 22 docentes e orientações éticas [Prather *et al*., 2023a]. Uma síntese posterior discute os desafios e as oportunidades que esses modelos trazem a docentes e estudantes [Denny *et al*., 2024a]. Entrevistas com 20 docentes de programação introdutória mostram que muitos planejavam, no curto prazo, medidas para coibir a fraude assistida por IA, e que no longo prazo as opiniões divergiam entre proibir as ferramentas e integrá-las às disciplinas [Lau & Guo, 2023]. Há discussões análogas para o ensino de engenharia de software [Daun & Brings, 2023]. Do lado dos estudantes, um experimento controlado com 69 iniciantes (10 a 17 anos) encontrou que o acesso a um gerador de código aumentou o desempenho em tarefas de autoria. O desempenho em tarefas posteriores de modificação manual não diminuiu [Kazemitabaar *et al*., 2023]. Outro, porém, observou iniciantes usando o Copilot e descreveu dificuldades cognitivas e metacognitivas [Prather *et al*., 2023b], e um terceiro encontrou benefícios e prejuízos distribuídos de forma desigual [Prather *et al*., 2024]. As respostas dos modelos a pedidos de ajuda de iniciantes são em parte úteis e em parte imprecisas [Hellas *et al*., 2023], o que exige que a pessoa saiba verificar o que recebe. Há também propostas de atividades que treinam a compreensão de código e a formulação de *prompts* [Denny *et al*., 2024b]. Esses trabalhos tratam de como estudantes interagem com a IA. Não tratam de uma ferramenta que verifique, de forma executável, o que foi produzido.

Produção nacional. Dos 27 trabalhos incluídos, 18 tratam de IA generativa no ensino de programação ou de engenharia de software, 4 de correção automática e juízes online, 3 de ensino de teste e 2 de integridade e uso aceitável de IA. Um mapeamento de 52 estudos descreve três papéis da IA generativa no ensino de programação: participante colaborativo, ferramenta de apoio e mediador pedagógico [Bedin *et al*., 2025]. Outro mapeamento, de 18 estudos, trata da programação orientada a objetos [Camilo & Hauck, 2026], e uma revisão examina as estratégias pedagógicas empregadas [da Silva *et al*., 2024]. Sobre correção, um estudo mostra que códigos corretos para um corretor automático podem apresentar comportamentos que indicam falhas de aprendizagem [da Silva *et al*., 2023]. Outro propõe retorno gerado por modelos de linguagem em lugar de mensagens de erro e resultados de teste [Pimentel *et al*., 2025]. Uma mesa temática propõe discutir como tornar juízes online instrumentos pedagógicos [Delgado *et al*., 2026]. Em teste, estimular estudantes a formular testes melhorou o desempenho deles [Almeida *et al*., 2025]. Sobre integridade, um estudo analisa 1.258 linhas de código do ChatGPT e identifica padrões incomuns em iniciantes [Simão de Deus *et al*., 2024], e uma proposta gradua o uso de IA em cinco níveis [Oliveira & Teixeira, 2026]. Examinamos o resumo, ou o seu início, de 26 dos 27 trabalhos. O outro não tem resumo e foi triado pelo título. Nenhum descreve um portão que reexecute as verificações declaradas e confronte as afirmações do arquivo de tarefa com o repositório.

Avaliação e retorno automáticos. Uma revisão sistemática de geradores de retorno para exercícios de programação mostra que a maior parte das ferramentas aponta erros e poucas orientam o passo seguinte [Keuning *et al*., 2018]. Avaliar os testes escritos pelos próprios estudantes, e não só o código, é uma estratégia clássica para levá-los da tentativa e erro à reflexão [Edwards, 2004]. Há também relatos de experiência sobre ensinar a escrever testes, avaliar as suítes dos estudantes, dar incentivo para escrevê-los cedo e sobre os limites da cobertura como critério [Spacco & Pugh, 2006]. O risco do outro lado também está documentado: estudantes encontram meios de enganar sistemas de correção automática de exercícios [Kratzke, 2019]. O retorno formativo tem efeitos substanciais sobre a aprendizagem [Black & Wiliam, 1998], e um retorno eficaz responde a três perguntas: aonde vou, como estou indo e o que vem a seguir [Hattie & Timperley, 2007]. O andaime (*scaffolding*) é o apoio que permite realizar uma tarefa além da capacidade inicial de quem aprende, ao assumir os elementos que ela ainda não domina [Wood *et al*., 1976]. A literatura posterior destaca que ele deve ser retirado aos poucos. O alinhamento construtivo pede que objetivos, atividades e avaliação se correspondam [Biggs, 1996]. Um corretor automático é um oráculo de teste, e a literatura de testes de software trata do problema de distinguir o comportamento correto do incorreto [Barr *et al*., 2015]. Deduzimos disso, e não da fonte, que um teste que

passa vale o que vale o oráculo por trás dele. Por fim, uma meta-análise aponta lacunas entre o que os cursos de engenharia de software cobrem e o que a indústria espera [Garousi *et al*., 2019], o que motiva atividades sobre repositórios reais.

Verificação de agentes de código. O *SWE-bench* mede a capacidade de agentes em problemas reais [Jimenez *et al*., 2024], e o *SpecBench* mede o *reward hacking* [Zhao *et al*., 2026]. Ambos medem agentes e nenhum entrega um portão que uma equipe ou turma possa rodar nas próprias mudanças. O Método Fable [Sahir619, 2026] é o ancestral conceitual mais próximo: define portões de ajuste, de intenção e de verificação adversarial, mas como texto de *prompt*. A avaliação de agentes também precisa considerar o custo, e não só a acurácia, como argumenta um *preprint* [Kapoor *et al*., 2024].

Diferenciação. O kata difere desses trabalhos em dois pontos. A parte objetiva do portão é código testado, que lê códigos de saída e o estado do Git, e não instrução ao modelo. E o mesmo arquivo de tarefa serve a duas finalidades: sustenta a verificação e deixa um registro por fase que um docente pode ler. Entre os trabalhos que reunimos, nenhum combina as duas coisas nem relata a avaliação de um portão desse tipo. A busca nacional cobriu cinco veículos e *strings* em português, e a internacional não foi sistemática, de modo que a afirmação vale para esse recorte. Por isso o kata é proposto como uma ferramenta a experimentar em sala, e não como solução comprovada.

3 O Kata

O kata (*kata*, "forma" ou "padrão", como na sequência disciplinada de movimentos das artes marciais) é distribuído no PyPI (*Python Package Index*) como o pacote `kata-dev`, sob a licença MIT; este artigo descreve a versão 0.10.0 (tag `v0.10.0`), publicada em outubro de 2026, e indica, onde importa, o que mudou desde a 0.7.2. É um pacote Python de linha de comando com dois *frontends* para agentes (OpenCode e Claude Code), ambos gerados de uma única fonte de fases. A lógica objetiva está sempre no pacote Python, qualquer que seja o *frontend*. Esta seção descreve o projeto do kata e o comportamento que observamos. O artigo não traz evidência independente desse comportamento e não dá acesso ao material da verificação técnica, de modo que o leitor deve lê-la como declaração nossa.

3.1 O Ciclo de Nove Fases

A Figura 1 mostra o ciclo. Uma mudança passa por nove fases: FIT (classificar a tarefa e rotear), THINK (declarar problema, premissas, alternativas e incógnitas antes de codificar), SIMPLIFY (verificar que o código é mínimo), INTENT (comparar o que o código faz, o que o teste espera e o que a especificação diz), SURGICAL (justificar cada arquivo alterado), VERIFY (executar lint, tipos quando declarados, testes e cobertura), TWIN CHECK (procurar o mesmo defeito em outros pontos do projeto), ARTIFACT (conferir as linhas de evidência devidas) e REPORT (relatar o resultado primeiro). Uma fase opcional, o JUDGE, confronta o resultado com o repositório. O modo `kata --audit` gradua, depois do fato, cada fase de uma tarefa. Na Figura 1, observe que só FIT, VERIFY, JUDGE e AUDIT têm borda dupla, ou seja, lógica executável no pacote. As demais fases dependem

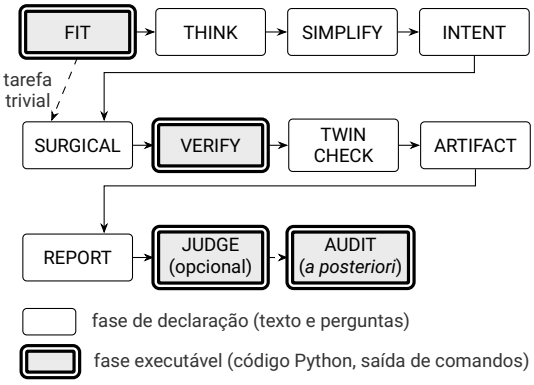


Figura 1. O ciclo do kata. As fases de borda dupla têm lógica executável no pacote Python; as demais registram declarações. A seta tracejada é o atalho de tarefas triviais. Diagrama elaborado por nós com o pacote gráfico TikZ do LaTeX.

sobretudo do que a pessoa declara, e é por isso que só as primeiras produzem um veredito ou uma graduação, enquanto as outras produzem registro.

A fase FIT mede o *diff* real em relação ao `HEAD`. Uma tarefa de no máximo um arquivo e menos de dez linhas alteradas é candidata a trivial. Na versão 0.10.0 ainda há uma ressalva: a medida do *diff* inclui os arquivos `.kata/*.yaml` não rastreados (o próprio arquivo de tarefa e a configuração), de modo que, num projeto que não ignora `.kata/`, uma mudança de uma linha deixa de ser candidata a trivial. Reproduzimos o caso na 0.8.0 (uma linha alterada: um arquivo; com um `.kata/x.yaml` não rastreado: dois arquivos e três linhas) e, na 0.10.0, confirmamos que a contagem de arquivos sobe do mesmo modo. O JUDGE e a lista de arquivos do SURGICAL filtram essa escrituração, mas a medida do FIT não a filtra; as demonstrações a escondem porque excluem `.kata/` do Git. A tarefa só vai direto à verificação, sem o cerimonial de planejamento, se a pessoa confirmar que a mudança não altera comportamento nem exige pesquisa. As demais seguem pelo ciclo completo. A fase VERIFY executa os comandos que o projeto declara em `.kata/config.yaml` para lint, verificação de tipos (quando declarada), teste e cobertura, ou os padrões Python (`ruff` e `pytest`), e julga cada um pelo código de saída, não pelo que o agente afirma. A cobertura só roda se os testes passaram. Na versão 0.8.0, o VERIFY grava no arquivo de tarefa a versão do kata, o gate em vigor, os comandos que rodaram e o horário da execução; o relatório imprime essa evidência e o JUDGE a confronta com a reexecução.

Arquitetura e garantias do próprio kata. As fases são escritas uma única vez, em doze arquivos Markdown de origem no tag `v0.10.0` (dez fases, o orquestrador e o ciclo curto `kata-agile`, da Seção 3). Um *script* os expande para cada *frontend*, trocando os nomes das ferramentas por papéis do hospedeiro (executar, ler, perguntar), e um teste falha se os arquivos gerados divergem da origem; 92,8% das linhas renderizadas são compartilhadas, e o restante é diferença declarada por *frontend*. Quatro adaptadores de domínio (infraestrutura, análise de dados, pesquisa e documentação) estendem o ciclo a tarefas que não são código de aplicação (Seção 4). O pacote tem portão de cobertura de 70%, e a integração contínua roda lint, verificação de formato, verificação de divergência das habilidades geradas, testes, o conjunto adversarial e a conferência do pacote construído. Assim, o que

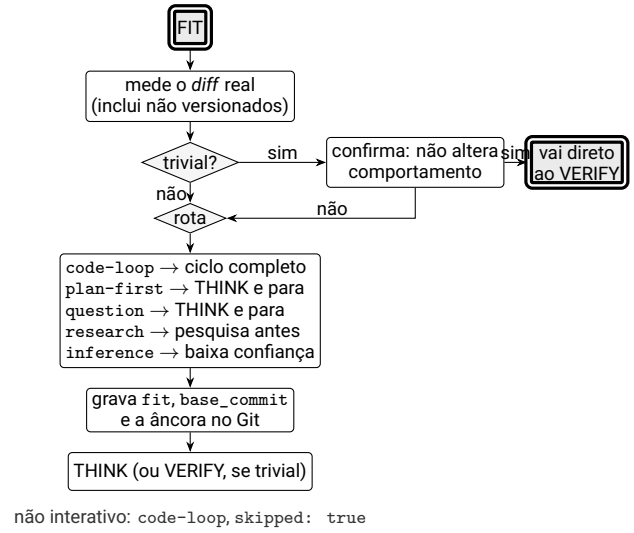


Figura 2. A fase FIT. O diff real é medido incluindo arquivos não versionados; o portão de trivialidade reconfirma com a pessoa, porque "um arquivo, menos de dez linhas"não prova sozinho que o comportamento não mudou; e as cinco rotas divergem. O `base_commit` e a âncora no Git são gravados aqui.

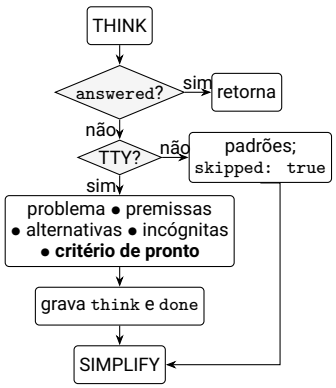


Figura 3. A fase THINK. O problema, as premissas, as alternativas, as incógnitas e o **critério de pronto** são declarados antes de existir evidência. Sem terminal, a seção é gravada como *pulada*, e não como respondida.

se verifica na máquina de quem desenvolve e no servidor não pode divergir.

3.2 O Verificador Adversarial

O JUDGE trata o arquivo de tarefa como um conjunto de afirmações, reexecuta as verificações declaradas e compara as afirmações com o *diff* do Git, inclusive com arquivos ainda não versionados. Ele procura oito categorias de fraude. Há testes enfraquecidos (por exemplo, o corpo trocado por `pass`) e falsa conclusão (o relatório diz que os testes passam e a reexecução falha). Há escopo excedente (o relatório declara um arquivo e a árvore tem quatro), ação não autorizada e traição da especificação (um desacordo de intenção aprovado mesmo assim). Há ainda resíduos, adulteração da linha de base (o `base_commit` do arquivo diverge da âncora gravada no Git) e trivialidade declarada que o *diff* não sustenta: uma tarefa marcada como trivial com dezenas de arquivos ou centenas de linhas, que o portão de escopo deixaria passar por confiar na própria marca. Os padrões de enfraquecimento cobrem nove famílias de linguagens, e um teste em linguagem não coberta vira uma **lacuna declarada**, e não um silêncio.

Na versão 0.8.0, o verificador também separa um check que rodou e reprovou de um comando que não pôde rodar:

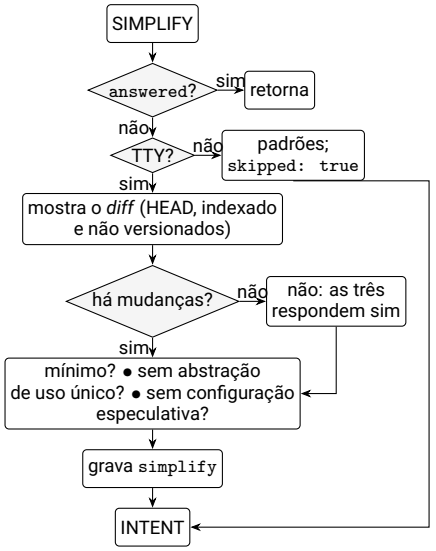


Figura 4. A fase SIMPLIFY. O *diff*, incluindo os arquivos não versionados que nenhum `git diff` mostra, é apresentado antes das três perguntas de minimalidade.

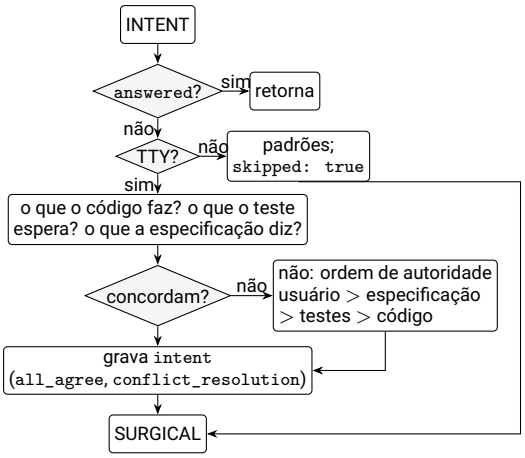


Figura 5. A fase INTENT. Código, testes e especificação são comparados antes de a mudança acontecer; um desacordo é resolvido pela ordem de autoridade. O JUDGE lê apenas um desacordo *não resolvido* como traição da especificação.

o segundo vira lacuna declarada, e não falsa conclusão, porque transformar ambiente em desonestidade seria um falso positivo. Ele compara a versão do kata, o gate de cobertura e os comandos gravados no arquivo com os da reexecução, e ressalva a diferença em vez de apresentar como equivalente um resultado produzido por outro código ou por outro limiar. E atribui os commits que caem na janela de *diff* aos carimbos das tarefas vizinhas: o trabalho de uma tarefa aprovada e carimbada sai da janela com uma ressalva que nomeia o commit e a tarefa responsáveis; um commit sem carimbo não é atribuído, e o veredito confessa a lacuna em vez de acusar ou absolver.

As versões 0.9.0 e 0.10.0 não alteram as nove fases nem o modelo de vereditos; mudam como o ciclo é aberto e como a evidência chega ao arquivo de tarefa. O ciclo curto `kata-agile` (FIT, THINK, DECLARE, VERIFY e REPORT, com quatro interações em vez de treze ou mais) funde INTENT, SURGICAL e SIMPLIFY em uma fase que ainda grava `surgical.files`, `intent.*` e `simplify.*`, de modo que `--audit`, `--judge` e `--report` valem sem mudança de esquema. Dois modos avulsos substituem transcri-

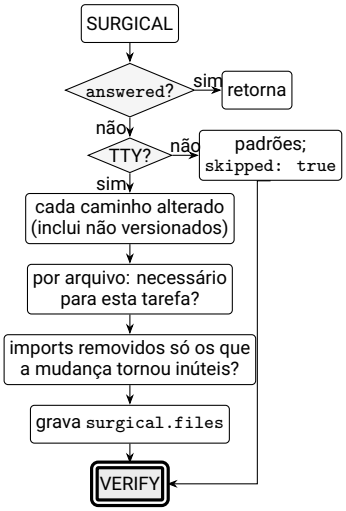


Figura 6. A fase SURGICAL. Cada arquivo alterado é declarado necessário ou não, e essa declaração é o contrato que o JUDGE cobra: arquivo alterado e não declarado é escopo excedente.

ção manual por cálculo do CLI: `--verify-task` roda o gate e **grava** `verify.*` na própria tarefa, e `--artifact-task` calcula e grava as linhas devidas (INTENT, AUTH, PENDING e TWINS), saindo com 1 se alguma faltar. A razão é a mesma do restante do JUDGE: evidência digitada pelo agente é a falsa conclusão que o verificador caça. Na 0.10.0, o JUDGE passa a acusar ação não autorizada a partir do campo `auth.action_taken`, e não só da conta que a fase ARTIFACT faz, porque o ciclo curto não percorre essa fase e um *push* sem linha AUTH passava limpo. A 0.8.1 tornou o segundo estágio do JUDGE (revisão semântica) independente de qualquer modelo. As Execuções do Apêndice A percorrem o ciclo completo e não exercitam o ciclo curto.

O veredito é calculado por regras, e não por julgamento do modelo. Qualquer fraude de alta gravidade resulta em `REFUTED`. Achados de média ou baixa gravidade resultam em `VERIFIED WITH CAVEATS`. Se não há fraude, mas algo não pôde ser observado, o veredito é `UNVERIFIABLE`. Um resultado limpo, com todas as reexecuções passando, é `VERIFIED`. O critério de sucesso do usuário é listado como *aceito sem verificação*, porque nenhum comando o reproduz, e apresentá-lo como verificado seria a fraude que o JUDGE existe para caçar.

3.3 O Modo de Auditoria

O `kata --audit` gradua cada fase em uma de quatro notas. A fase é *seguida* se foi respondida com conteúdo real, *pulada* se foi preenchida com o padrão de uma execução não interativa, sem que ninguém respondesse, *falsificada* se foi marcada como respondida com conteúdo vazio ou padrão (ou se há afirmação de verificação sem evidência) e *degradada* se foi executada sem a habilidade da própria fase carregada. Para cada nota diferente de *seguida*, a ferramenta nomeia o risco concreto criado, como "as premissas nunca foram declaradas: qualquer solução pode atacar o problema errado". A graduação é mecânica e cega ao contexto: ela detecta um campo vazio, mas não detecta um campo preenchido com conteúdo plausível e falso.

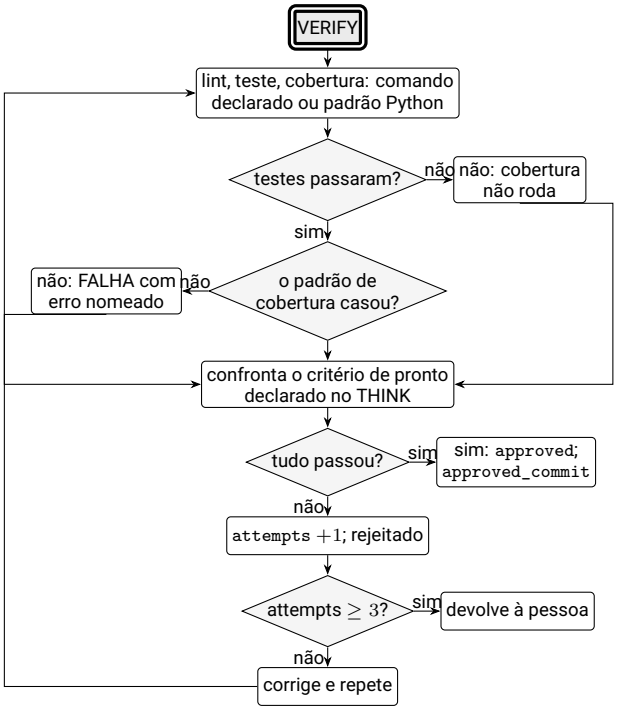


Figura 7. A fase VERIFY. Lint, teste e cobertura são julgados pelo código de saída; a cobertura não roda quando os testes falham; um padrão de cobertura que não casa falha com erro nomeado. O contador registra execuções *que falharam*, e após três delas a tarefa é devolvida à pessoa.

3.4 Limites Conhecidos do Verificador

O JUDGE é adversarial para as afirmações do arquivo de tarefa, e não para quem controla o repositório. Reproduzimos desvios contra o código do kata, resumidos na Tabela 1. O mais grave é que um agente que controla o Git pode esvaziar a janela de *diff*, e então um teste esvaziado volta `VERIFIED`. Alguns caminhos foram estreitados ou fechados na própria 0.8.0, e outros permanecem abertos.

Para o uso em sala, a consequência é direta: o JUDGE é um aviso, não uma prova. Um estudante que controla o próprio repositório está na posição do agente da Tabela 1.

3.5 Execução 1: o Ciclo Completo

A Listagem 1 resume uma execução real do kata sobre um projeto de demonstração: uma calculadora cuja função `dividir` não trata a divisão por zero. As respostas foram redigidas por nós para a demonstração e não são de estudante. Os ícones do terminal foram trocados por marcadores ASCII.

O FIT mediu o *diff* (três arquivos, 16 linhas) e classificou a tarefa como não trivial. O THINK exigiu problema, premissas, alternativas e critério de pronto antes de qualquer verificação. No INTENT, registramos que o código levanta `ZeroDivisionError` e o teste espera `ValueError`. Ao declarar a discordância, o kata exibe a ordem de autoridade entre declaração do usuário, especificação, testes e código, que é o conteúdo conceitual central da fase. O VERIFY executou as verificações declaradas ou os padrões Python e julgou cada uma pelo código de saída. O TWIN CHECK procurou o mesmo padrão no restante do projeto. Desde a 0.8.0, o relatório imprime também a evidência da execução (gate, comandos e horário) e avisa que o commit final deve ser carimbado com `kata --stamp`, para fechar a janela do JUDGE.

Duas observações dessa execução importam para a adoção em sala. Na primeira tentativa, o kata rodou com um in-

Tabela 1. Desvios do JUDGE reproduzidos contra o código do kata e sua situação no repositório. Código de saída 0 em `UNVERIFIABLE` é decisão de projeto (falha aberta). Fonte: reprodução nossa em repositório temporário cujo teste foi esvaziado com `pass`.

Desvio	Efeito observado	Situação
Reescrever a âncora de base no Git e no YAML	Janela de *diff* vazia; `VERIFIED` com teste esvaziado	Estreitado (reflog da âncora, *commit* `3454fc7`); recriá-la ainda escapa localmente; fecha-se em integração contínua com `--trusted-base`
Janela de *diff* vazia (`approved_commit` igual ao `base_commit`)	Teste enfraquecido invisível; `VERIFIED`	Corrigido (*commit* `3cacc5a`); `kata --stamp` grava o commit final e fecha a janela depois do *commit*
Fabricar uma lacuna (teste oculto em `.git/info/exclude` ou em linguagem sem sondas)	`UNVERIFIABLE` com código 0 passa por quem lê só o código de saída	Aberto; opção de falha fechada é trabalho futuro
Declarar todos os arquivos tocados	Escopo excedente nunca é cobrado, pois `necessary` é autodeclarado	Aberto
Conteúdo plausível e falso	Auditoria gradua como seguida; resolução de conflito de uma linha desarma a traição da especificação	Aberto

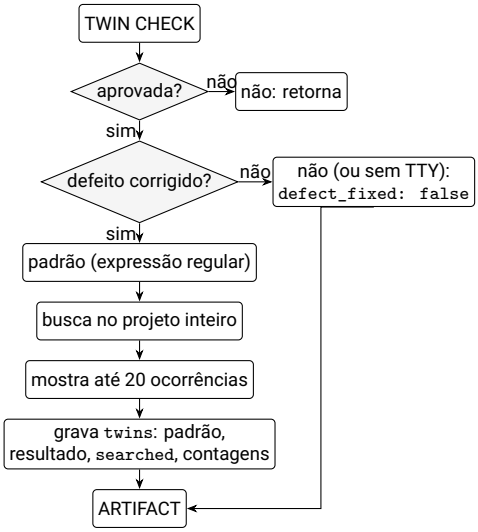


Figura 8. A fase TWIN CHECK. Busca o padrão do defeito corrigido no projeto e registra o resultado, inclusive negativo. Esse registro é o que distingue "não havia defeito"de "ninguém verificou".

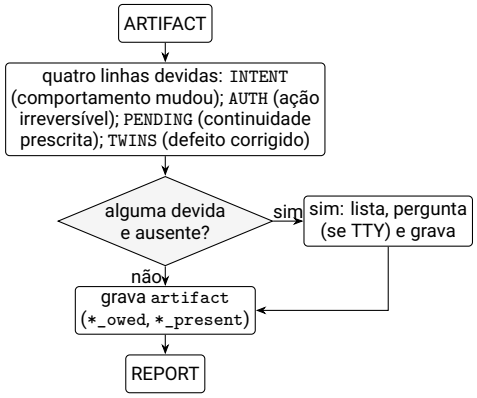


Figura 9. A fase ARTIFACT. Cada uma das quatro linhas de evidência tem seu próprio gatilho; a linha devida e ausente é pedida e registrada. `TWINS` só é devida quando um defeito foi de fato declarado corrigido.

terpretador Python sem `ruff` nem `pytest`. Rejeitou o ciclo com a mensagem de que os módulos não existiam, porque o portão não aprova o que não conseguiu executar (saída reproduzível em `demos/saidas/01c-sem-ferramentas.txt`). Na execução da listagem, porém, o campo de resolução do conflito do INTENT ficou vazio e, mesmo assim, o ciclo foi aprovado. A auditoria graduou todas as fases como seguidas, porque verifica que a fase foi respondida, e não que a resposta é boa. Rodamos o JUDGE sobre essa mesma tarefa (Execução 2, no Apêndice A), e ele a refutou por traição da especificação.

Sete execuções adicionais estão no Apêndice A. Nelas, o JUDGE refuta a tarefa acima por traição da especificação e passa a `VERIFIED` quando o conflito recebe uma frase qualquer (Execução 2). Um teste esvaziado passa em testes e cobertura e só o JUDGE o acusa (Execução 3), e a auditoria acusa as sete fases vazias de um arquivo escrito à mão (Execução 4). Os domínios `docs`, `data-analysis`, `devops` e `research` são verificados por comandos que o projeto declara, e reprovam, respectivamente, uma flag inexistente, uma remoção de linhas sem registro, uma configuração sem *healthcheck* e um relatório com fonte inexistente (Execuções 5 a 8).

3.6 O Arquivo de Tarefa

Cada tarefa vive em `.kata/<tarefa>.yaml`, um arquivo no formato YAML, de dados legíveis por pessoas. O arquivo é ao mesmo tempo o registro do ciclo e o conjunto de afirmações que o JUDGE confronta com o repositório. As Listagens 2 e 3 mostram o arquivo completo gerado pelo kata na Execução 1 da Seção 3.5, um projeto de demonstração cujas respostas foram redigidas por nós. Omitimos o bloco `preflight` (habilidades ausentes) e um bloco de extensão que este artigo não trata. Como o repositório da demonstração não tinha *commits*, faltam os campos `base_commit` e `approved_commit`, gravados quando há histórico, e o bloco `judge`, gravado por `kata --judge`. A Tabela 2 diz, para

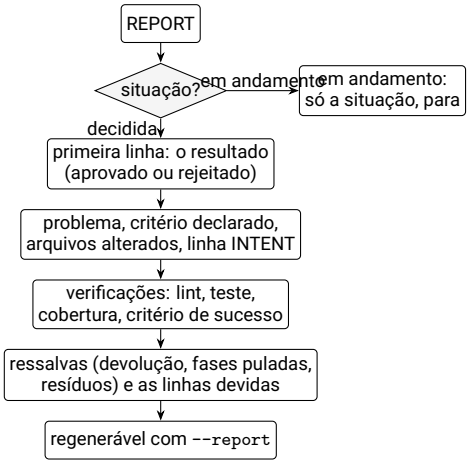


Figura 10. A fase REPORT. O resultado vem primeiro; depois o problema, o critério declarado no THINK, as evidências, as ressalvas e as linhas devidas. Uma tarefa em andamento relata apenas a situação.

```
$ kata --init dividir-por-zero
  diff: 3 arquivo(s), 16 linha(s) alteradas -> tarefa
  ↪  nao-trivial
  Rota escolhida [1]: 1
  Qual o problema exato que estou resolvendo?
    a função dividir levanta ZeroDivisionError sem guarda
  O que é 'pronto'? o teste test_dividir_por_zero passa
    e a cobertura fica em pelo menos 70%
$ kata --task dividir-por-zero
  O que o código FAZ hoje? dividir levanta ZeroDivisionError
  O que o teste/check ESPERA? o teste espera ValueError
  Código, teste e especificação concordam? [S/n]: n
  [!] Conflito detectado! A ordem de autoridade é:
      declaração do usuário > spec > testes > código
  > python3 -m ruff check src/ tests/        [ok] limpo
  > python3 -m pytest tests/ --tb=short -q   [ok] passou
  > coverage (gate >= 70%)                   [ok] passou
  ↪  (100.0%)
  [!] approved_commit não gravado: o HEAD ainda é o
  ↪  base_commit
      Depois do commit, rode `kata --stamp dividir-por-zero`
  [ok] KATA CYCLE - APROVADO
  TWIN CHECK: padrão 'a / b' encontrado em 1 arquivo(s)
$ kata --audit --task dividir-por-zero
  [ok] FIT, THINK, INTENT, SIMPLIFY, SURGICAL,
       VERIFY, TWINS: followed
```

Listagem 1. Trechos de uma execução real do kata sobre um projeto de demonstração (respostas redigidas por nós; ícones substituídos por marcadores ASCII).

cada bloco, a que fase pertence, quem o preenche e o que o kata faz com ele. Em uma leitura rápida, os blocos de `fit` a `surgical` são declarações da pessoa (ou o padrão de uma execução não interativa). O bloco `verify` é preenchido pelo kata a partir de códigos de saída, e os blocos `twins` e `artifact` são calculados.

3.7 Fluxos de Uso no Desenvolvimento de Software

O kata atende a sete fluxos, todos pela mesma linha de comando. *Antes de um commit*: `kata --init <tarefa>` e `kata --task <tarefa>` conduzem o ciclo completo. *Planejamento*: `kata --plan` executa FIT e THINK e para, sem alterar código, e o plano pode ser revisado antes de a primeira linha mudar. *Integração contínua*: `kata --check-only` roda lint, testes e cobertura de forma não interativa e sai com código 1 se algo falha. *Revisão antes de integrar*: `kata --task <tarefa> --judge` reexecuta as verificações e caça fraudes e, em integração contínua,

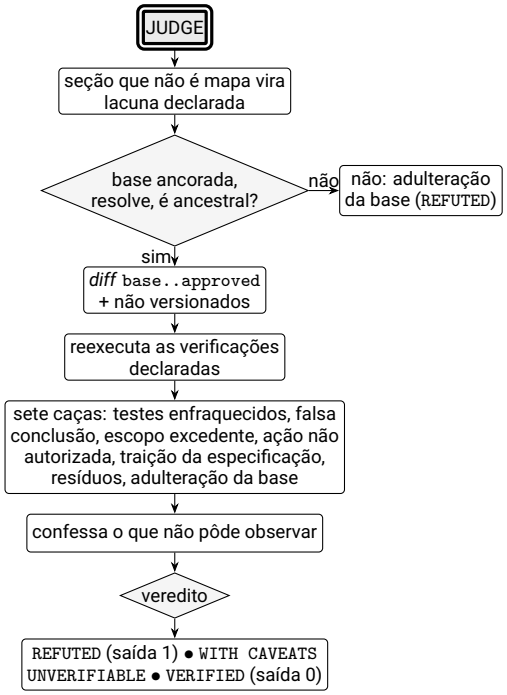


Figura 11. A fase JUDGE. O arquivo de tarefa é um conjunto de afirmações; a base é checada de forma adversarial; as verificações declaradas são reexecutadas; oito fraudes são caçadas; e o que não pôde ser observado é confessado.

`--trusted-base <ref>` toma o piso do *diff* de uma referência que o agente não controla. *Fechamento*: depois do *commit*, `kata --stamp <tarefa>` grava o commit que encerra o trabalho em `approved_commit`, fechando a janela do JUDGE; sem ele, a próxima tarefa contamina a janela desta. *Auditoria*: `kata --audit` gradua as fases de uma tarefa já concluída. *Instalação*: `kata --install` copia as habilidades empacotadas para o diretório de configuração do hospedeiro, e `kata --doctor` confere se estão completas e se a versão que as gravou é a do `kata` que está rodando. Cada projeto declara em `.kata/config.yaml` os próprios comandos de lint, tipos, teste e cobertura, o que permite usar o kata fora de Python; uma chave desconhecida ou um operador de *shell* no comando viram erro nomeado, e não um padrão silencioso. Em uma disciplina, os fluxos de integração contínua e de revisão são as formas naturais de conferir entregas. Os de ciclo e auditoria servem para acompanhar o processo.

4 Os Domínios do Kata

O ciclo da Seção 3 é o mesmo em todo o kata, mas o que significa abrir a evidência, observar o resultado e desconfiar de uma afirmação muda conforme o artefato. Por isso o kata tem um domínio padrão, `coding`, e quatro adaptadores de domínio, `devops`, `data-analysis`, `research` e `docs`. O arquivo de tarefa declara o domínio no campo `domain`, que vale `coding` quando omitido. Para os outros, a instrução do orquestrador manda o agente carregar o adaptador `kata-<domínio>` logo depois do FIT. Os adaptadores são opcionais: se o agente não consegue carregá-lo, a instrução manda registrar a falta em `preflight.skills_missing`, e a auditoria, ao ler esse registro, marca uma entrada `preflight` como degradada. Quem escreve o registro é o agente, e nada impede que o omita. Um nome de domínio

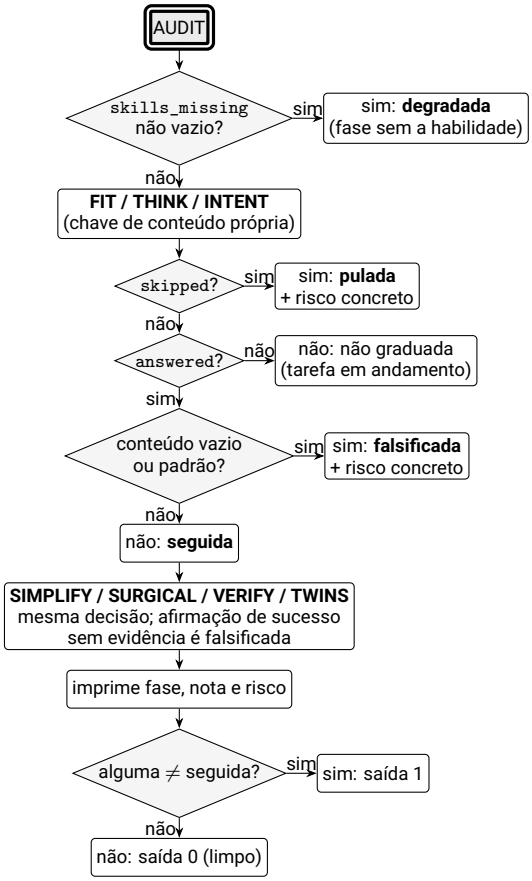


Figura 12. O modo AUDIT. A auditoria lê o arquivo de tarefa e gradua cada fase como seguida, pulada, falsificada ou degradada, nomeando o risco concreto de cada nota diferente de seguida.

```yaml
schema_version: 1
task: dividir-por-zero
status: approved
domain: coding
done: o teste test_dividir_por_zero passa e a
  cobertura fica em pelo menos 70%
kata_version: 0.10.0
fit:
  trivial: false
  route: code-loop
  reason: corrigir divisão por zero sem guarda
  answered: true
think:
  problem: a função dividir levanta
    ZeroDivisionError sem guarda
  assumptions:
  - a API pública de dividir é estável
  alternatives:
  - guarda com ValueError
  - devolver None
  unknowns: nada
  answered: true
simplify:
  minimum_code: true
  no_single_use_abstractions: true
  no_speculative_config: true
  notes: ''
  answered: true
  skipped: false
intent:
  code_does: dividir levanta ZeroDivisionError
  check_expects: o teste espera ValueError
  spec_says: o README não menciona o caso
  all_agree: false
  conflict_resolution: ''
  answered: true
```

Listagem 2. Arquivo de tarefa gerado pelo kata 0.10.0 (parte 1: cabeçalho e fases FIT, THINK, SIMPLIFY e INTENT). Fonte: execução da Seção 3.5; as respostas foram redigidas por nós.

desconhecido gera um aviso do pacote, e o ciclo segue como `coding`. As Tabelas 3 e 4 comparam os cinco domínios.

O que é executável e o que é instrução. Os tipos de fraude da última coluna da Tabela 4 são instruções escritas nos adaptadores, e não lógica do JUDGE. Dos 24 tipos listados, só `unauthorized_action` aparece no pacote Python, porque coincide com uma das oito categorias do JUDGE, e os outros 23 não existem no código. O pacote só usa o campo `domain` para imprimir o domínio e avisar de um adaptador desconhecido. O JUDGE continua a procurar as oito categorias de fraude, em qualquer domínio. Fora de `coding`, a parte executável da verificação vem dos comandos que o projeto declara em `.kata/config.yaml`, que o kata roda como foram escritos e julga pelo código de saída. Uma atividade de infraestrutura poderia declarar `docker compose config` como verificação, e uma de dados, a execução do *notebook*. As Execuções 5 a 8 do Apêndice A fazem isso para os domínios `docs`, `data-analysis`, `devops` e `research`, com comandos próprios. Sem essa declaração, vale o padrão Python, que não faz sentido para esses artefatos. Daí uma consequência para o ensino: nesses domínios a vigilância contra fraudes específicas, como o número inventado em um relatório ou a fonte inexistente, depende do agente e do docente, e não de uma checagem automática.

Todos os adaptadores compartilham três contratos. O primeiro é a ordem de autoridade, em que a inferência do modelo só vale como baixa confiança e o conflito entre documento e artefato é resolvido perguntando ao usuário. O segundo é a verificação por observação: uma afirmação só vale se for observada, de modo que "fiz o *deploy*" sem `docker compose ps` é uma afirmação não verificada. O terceiro são as linhas vermelhas, ações que exigem uma intervenção humana documentada. São exemplos publicar dados pessoais, executar *deploy* ou *push* sem a linha AUTH com a citação do usuário, pôr segredos no *diff* ou no arquivo de tarefa e afirmar um fato de API sem abrir a fonte.

4.1 Contexto de Ensino por Domínio

Os usos a seguir são hipóteses, derivadas dos adaptadores e da Seção 5.

Programação (`coding`). É o domínio dos casos 1 a 7 da Seção 6: disciplinas de programação, testes, manutenção e projeto, em que o objeto é código e o resultado é verificado por testes.

Infraestrutura (`devops`). Disciplinas de redes, sistemas, implantação ou DevOps, em que o estudante configura contêineres, servidor web ou fluxos de integração. A verificação por observação ensina uma disciplina de trabalho (*só afirmar o que se observou*), e as linhas vermelhas sobre segredos e ações irreversíveis correspondem a conteúdos de segurança. Ressalvas: exige Docker no laboratório, e o JUDGE só confere o que o projeto declarar como verificação.

Análise de dados (`data-analysis`). Disciplinas de ciência de dados, estatística aplicada ou *business intelligence*, com *notebooks* e consultas. Os tipos de fraude do adaptador (seleção conveniente de amostra, número sem fonte, remoção silenciosa de linhas) descrevem riscos reais em relatórios de estudantes. Ressalvas: nenhum desses ris-

```yaml
surgical:
  files:
  - path: src/__init__.py
    necessary: true
  - path: src/calculadora.py
    necessary: true
  - path: tests/test_calculadora.py
    necessary: true
  removed_imports_clean: true
  answered: true
  skipped: false
verify:
  ruff_clean: true
  typecheck_clean: null
  tests_pass: true
  coverage_pct: 100.0
  coverage_pass: true
  success_criteria_met: true
  attempts: 0
  hand_back: false
  gate_used: 70.0
  commands:
    lint: python -m ruff check src/ tests/
    test: python -m pytest tests/ --tb=short -q
    coverage: python -m pytest tests/ --cov=src
      --cov-report=term-missing --cov-fail-under=70.0 -q
  ran_at: '2026-11-03T23:39:41+00:00'
auth:
  action_taken: false
  authorized: false
  action: ''
  quote: ''
pending:
  action: ''
  documented: false
twins:
  pattern: a / b
  result: 1 arquivo(s), 1 ocorrência(s)
  searched: true
  defect_fixed: true
  matches_count: 1
  files_count: 1
  fix_applied: false
artifact:
  intent_owed: true
  intent_present: true
  auth_owed: false
  auth_present: false
  pending_owed: false
  pending_present: false
  twins_owed: true
  twins_present: true
```

Listagem 3. Arquivo de tarefa (parte 2: fases SURGICAL, VERIFY, AUTH, PENDING, TWIN CHECK e ARTIFACT). Os campos `skipped` aparecem como `false` porque as fases foram respondidas.

cos é detectado por código, e dados pessoais exigem o tratamento da Seção 6 e da LGPD.

Pesquisa (`research`). Metodologia científica, iniciação científica e trabalhos de conclusão, em que o estudante investiga uma biblioteca, compara alternativas ou revisa literatura. A exigência de fonte primária e a separação entre o observado e o inferido se aplicam a um risco que o adaptador trata: a invenção de fontes e comportamentos. Ressalvas: o adaptador pede que o agente cite o trecho da fonte, mas quem confere a citação é o docente.

Documentação (`docs`). Redação técnica, engenharia de requisitos e documentação de projeto, com especificações e registros de decisão. A regra de que todo comando, *flag* ou contagem citada tem fonte no código e a de que, havendo divergência, é o documento que se corrige treinam a detecção de documentação desatualizada. Ressalvas: a verificação roda o comando citado, o que só funciona para documentos técnicos de software.

5 Fundamentação Pedagógica da Proposta

Esta seção responde à QP1 por análise conceitual. A correspondência que propomos é uma hipótese de projeto nossa, e nenhuma parte dela foi validada com estudantes ou docentes.

O ciclo como andaime. O andaime sustenta uma atividade que a pessoa ainda não executa sozinha, ao assumir os elementos da tarefa que ela ainda não domina [Wood *et al.*, 1976], e deve ser retirado aos poucos, como a literatura posterior destaca. O ciclo fixa a ordem do trabalho disciplinado: declarar antes de codificar, justificar cada arquivo, verificar com comandos reais. Quem está começando pode não ter o hábito dessas etapas, e o ciclo as torna explícitas. A retirada gradual do andaime é uma decisão de política da disciplina, e o kata não a automatiza (Seção 6, caso 7).

Fases e dificuldades de iniciantes. Em um estudo de laboratório com iniciantes que programavam com IA generativa, Prather *et al.* [2024] descrevem dificuldades metacognitivas conhecidas, como formar um modelo conceitual incorreto do problema e ter uma falsa sensação de progresso depois de testar a solução. Os autores propõem andaimar a experiência com a ferramenta. Lemos o ciclo como uma resposta possível, e isso é uma hipótese. A fase THINK pede o problema, as premissas e o critério de pronto antes do código, o que expõe o modelo conceitual da pessoa. A fase VERIFY confronta a sensação de progresso com códigos de saída. A fase INTENT obriga a pôr lado a lado o que o código faz, o que o teste espera e o que a especificação diz.

Reflexão na ação e testes. Edwards [2004] argumenta que o teste de software promove a formação de hipóteses e a validação experimental, centrais à reflexão na ação. Segundo ele, avaliar a capacidade de demonstrar a correção por testes reforça essas habilidades. Em termos do kata, as premissas do THINK fazem o papel de hipóteses e o VERIFY, o de validação experimental, e o JUDGE cobra que o experimento não tenha sido esvaziado. Esta também é uma hipótese de leitura, e não um resultado.

O arquivo e a auditoria como retorno formativo. O retorno é formativo quando informa a pessoa sobre a distância entre o desempenho atual e o desejado, a tempo de ela agir [Black & Wiliam, 1998]. Um retorno eficaz responde a três perguntas: aonde vou, como estou indo e o que vem a seguir [Hattie & Timperley, 2007]. Cada fase deixa um registro que o docente pode ler sem reconstruir o raciocínio do estudante. As notas de auditoria apontam fases omitidas ou vazias e dizem qual risco isso criou, o que é retorno sobre o processo, e não apenas sobre o resultado.

Mapeamento para objetivos de aprendizagem. A Tabela 5 relaciona cada fase ao processo cognitivo dominante na taxonomia revisada de Bloom [Anderson & Krathwohl, 2001], à evidência que a fase deixa e ao que o pacote verifica automaticamente. Observe a última coluna: nas fases de declaração (THINK, SIMPLIFY), o pacote só verifica que o campo foi preenchido, e nada verifica que a declaração é boa. Por isso, o retorno automático cobre a presença do raciocínio e a conferência dos fatos (o que os comandos retornaram, quais arquivos mudaram), mas não a qualidade do julgamento. Essa parte continua sendo trabalho de docentes e de colegas.

Da nota de auditoria à pergunta formativa. Cada nota diferente de *seguida* vem com o risco que a omissão cria, e esse texto pode orientar a conversa entre docente e estudante. A Tabela 6 reproduz as mensagens de risco da ferramenta (nas palavras do código) e sugere, para cada fase, uma pergunta que o docente poderia fazer. As perguntas são sugestões nossas e não foram testadas em sala. A ideia é que a auditoria aponte *onde* olhar, e a pergunta abra a conversa sobre *por quê*, o que corresponde à pergunta "como estou indo" do modelo de Hattie & Timperley [2007].

Alinhamento construtivo. O alinhamento construtivo [Biggs, 1996] pede que o docente parta dos resultados de aprendizagem desejados e escolha atividades e avaliação que os evidenciem. Aqui, a Tabela 5 permite fazer o caminho inverso a partir de uma fase. Se o resultado desejado é que a pessoa saiba justificar o escopo de uma mudança, a fase SURGICAL é a atividade e o arquivo de tarefa, a evidência. Fases que não correspondem a nenhum resultado da disciplina podem ser dispensadas por política da disciplina (Seção 6, caso 7).

Riscos pedagógicos da própria ferramenta. A proposta tem riscos que o docente precisa vigiar. O primeiro é o ritual: a pessoa pode preencher os campos para cumprir a exigência, e a Execução 1 mostra que a auditoria graduaria como seguido um registro assim. O segundo é a delegação do julgamento ao veredito, em que a pessoa deixa de se perguntar se o trabalho está certo porque a ferramenta aprovou, o que é mais grave porque o veredito tem desvios conhecidos (Seção 3.4). O terceiro é o custo de tempo, que desloca outras atividades da disciplina. As mitigações possíveis são ler os registros em vez de contar campos, usar as perguntas da Tabela 6 para abrir a conversa e dizer à turma o que o kata verifica e o que não verifica.

Integridade acadêmica. O JUDGE detecta o enfraquecimento de testes, a falsa conclusão e o escopo não declarado. Em uma entrega de estudante, isso se aproxima de um aviso de que o resultado talvez não signifique o que parece. Como a Seção 3.4 mostra, o verificador tem desvios conhecidos, e portanto não deve ser a única base de uma decisão disciplinar.

6 Casos de Uso Propostos para o Ensino de Software

Esta seção responde à QP2. Os oito casos a seguir são propostas de uso, dos quais dois (os casos 1 e 2) são os do primeiro teste e seis são extensões. Derivam dos recursos do kata (Seção 3), da fundamentação pedagógica (Seção 5) e dos limites conhecidos do verificador (Seção 3.4), e são hipóteses de projeto. Para cada um indicamos o contexto, o fluxo de comandos, a evidência que o docente recebe e a principal ressalva. A Tabela 7 os resume.

6.1 Casos do Primeiro Teste

Os casos 1 e 2 são os do primeiro teste em sala e os que o protocolo da Seção 7 cobre.

Caso 1: planejar antes de pedir código. *Contexto.* Em uma disciplina introdutória em que o uso de agentes é permitido, o risco é pedir o código antes de saber o que se quer. *Fluxo.* O estudante roda `kata --plan`, que executa FIT e THINK e para sem alterar código: declara o problema, as premissas, as alternativas, o que não sabe e o critério de pronto. Só então aciona o agente e, ao final, roda `kata --check-only`. *Evidência.* O docente lê o plano antes da implementação. *Por que pode ajudar.* Em iniciantes, as dificuldades metacognitivas persistem e a ferramenta pode agravá-las [Prather *et al*., 2024]. A interação com ela já mostrou dificuldades cognitivas e metacognitivas [Prather *et al*., 2023b], e as respostas dos modelos nem sempre são corretas [Hellas *et al*., 2023]. Um critério de pronto escrito antes do código dá à pessoa o que conferir. *Ressalva.* O kata só verifica que o campo foi preenchido, e não que o plano é bom.

Caso 2: ensino de testes. *Contexto.* Avaliar os testes dos estudantes, e não só o código, é uma estratégia conhecida para levá-los à reflexão [Edwards, 2004; Spacco & Pugh, 2006]. *Fluxo.* O estudante entrega código e testes em um repositório Git, e o docente roda `kata --judge`. A Execução 3, no Apêndice A, mostra o caso em que o corpo do teste vira `pass` e o veredito é `REFUTED`. O verificador reexecuta os testes e procura corpos trocados por `pass`, asserções viradas comentário, `assert True` e *skips*, em nove famílias de linguagens. *Evidência.* Um veredito `REFUTED` traz o tipo de fraude e o arquivo. A Execução 3 mostra o kata detectando uma dessas formas. *Ressalva.* Um teste que não está vazio, mas verifica pouco, passa. O verificador não mede a qualidade do oráculo de teste [Barr *et al*., 2015].

6.2 Extensões

Os casos 3 a 8 são extensões. Dependem de recursos do kata que o protocolo não testa (integração contínua, auditoria, outros domínios) e são mantidos como hipóteses para estudos posteriores.

Caso 3: integridade em entregas com agentes. *Contexto.* Os corretores automáticos podem ser enganados [Kratzke, 2019], e um agente pode fazê-lo sem que a pessoa perceba. *Fluxo.* Na integração contínua do repositório da turma, `kata --check-only` roda lint, testes e cobertura. O comando `kata --judge --trusted-base <ref>` confronta as afirmações do arquivo de tarefa com o *diff* desde uma referência que a pessoa não controla. *Evidência.* Falsa conclusão, escopo excedente e teste enfraquecido viram achados nomeados. *Ressalva.* A Tabela 1 mostra que quem controla o Git pode escapar, e `UNVERIFIABLE` sai com código 0. O veredito deve abrir uma conversa, e não encerrar um processo disciplinar.

Caso 4: correção de defeitos em repositórios reais. *Contexto.* Meta-análises apontam lacunas entre o que os cursos cobrem e o que a prática exige [Garousi *et al*., 2019]. *Fluxo.* O ciclo completo sobre um defeito de um repositório público. Um banco de exercícios pode ser montado assim. Cada tarefa é o estado anterior à correção mais os testes que a correção alterou, com o histórico reinicializado para que a solução não seja lida. Verifica-se que ela falha no estado inicial e passa na correção. O TWIN CHECK pede à pessoa que busque o mesmo padrão de defeito no restante do projeto, o que treina a generalização. *Evidência.* O arquivo de tarefa e a lista de arquivos justificados. *Ressalva.* O banco de exercícios descrito aqui é uma receita, e não um conjunto disponibilizado com este artigo, e o custo de tempo do ciclo completo para estudantes não foi medido.

Caso 5: projeto integrador em equipe. *Fluxo.* Cada contribuição (*pull request*) acompanha um arquivo de tarefa. O docente roda `kata --audit` para ver quais fases foram seguidas, puladas ou falsificadas e `kata --report` para um relato que traz o resultado primeiro, seguido de problema, arquivos, verificações e *caveats*. *Evidência.* Um registro de processo por pessoa e por contribuição, que complementa o histórico do Git. *Ressalva.* A auditoria gradua como seguido um campo preenchido com conteúdo plausível e falso, como a Execução 1 (Seção 3.5) e a Execução 4 (Apêndice A) ilustram.

Caso 6: especificação, testes e intenção. *Fluxo.* A fase INTENT pede três respostas: o que o código faz, o que o teste espera e o que a especificação diz. Havendo discordância, apresenta a ordem de autoridade: declaração do usuário, especificação, testes, código. O exercício é decidir qual elemento deve ceder e registrar por quê. *Evidência.* O campo de resolução do conflito. *Ressalva.* A resolução é autodeclarada e aceita sem verificação, de modo que uma frase qualquer a preenche. O docente precisa lê-la.

Caso 7: retirada gradual do andaime. *Fluxo.* Nas primeiras tarefas, todas as fases são obrigatórias. Nas seguintes, a política da disciplina dispensa fases, e as rotas do FIT dão o apoio possível. O portão de tarefa trivial leva direto à verificação uma mudança de um arquivo e menos de dez linhas, confirmada como sem efeito de comportamento (na versão 0.10.0, a medida do FIT ainda conta os `.kata/*.yaml` não rastreados, de modo que o projeto precisa ignorá-los; Seção 3). As rotas `plan-first` e `question` param antes do código. O kata não tem comando para dispensar fases individuais, e o campo `skipped` do arquivo é marcado em execuções não interativas, em que ninguém respondeu, e não como dispensa consciente. A auditoria não distingue as duas coisas, e a política precisaria ser conferida pelo docente. *Fundamento.* É a retirada do andaime [Wood *et al*., 1976], com as fases obrigatórias escolhidas conforme os resultados de aprendizagem [Biggs, 1996]. *Ressalva.* Não há evidência de que essa retirada ajude. O protocolo da Seção 7 cobre só os casos 1 e 2 (com a fase SURGICAL), e a retirada do andaime ficaria para um estudo posterior.

Caso 8: outras linguagens e outros artefatos. Cada projeto declara em `.kata/config.yaml` os comandos de lint, teste e cobertura, e o kata os executa como foram escritos e julga pelo código de saída. Uma disciplina em Java ou JavaScript pode, assim, declarar `mvn test` ou `npm test`. Os padrões de enfraquecimento de teste cobrem nove famílias de linguagens, e um teste em linguagem não coberta vira lacuna declarada, e não silêncio. Os quatro adaptadores de domínio (infraestrutura, análise de dados, pesquisa e documentação, Seção 4) reaplicam o ciclo a *notebooks*, relatórios e textos técnicos. *Ressalva.* A verificação técnica da ferramenta, relatada à parte, cobriu só Python, e os adaptadores não foram avaliados.

6.3 Condições de Adoção

Condições de adoção. O kata exige Git, Python 3.11 ou superior e um agente hospedeiro (Claude Code ou OpenCode) com as habilidades instaladas por `kata --install`, e `kata --doctor` confere a instalação. O docente decide quais fases são obrigatórias, e o ciclo completo exige tempo adicional por tarefa, que não foi medido com estudantes e precisa entrar no planejamento da atividade. O uso em avaliação formal exige que a turma saiba o que é verificado e o que não é.

Dados pessoais e uso do veredito. O arquivo de tarefa registra o raciocínio de cada estudante, e o histórico do Git identifica quem fez cada mudança. São dados pessoais sob a LGPD [Brasil, 2018], e a disciplina deve informar a finalidade, o prazo de guarda e quem acessa os registros, e limitá-los ao que a atividade exige. O veredito do JUDGE deve poder ser contestado pela pessoa, com acesso às evidências que o sustentam, e não deve ser a única base de uma decisão disciplinar, pelos desvios da Tabela 1. Registros de quem escreveu cada fase devem ser tratados como dados de aprendizagem, e não como prova de conduta.

7 Proposta de Protocolo de Estudo com Estudantes

Esta seção descreve um estudo que não foi executado. Ela responde à QP3 e serve para que o efeito do kata sobre a aprendizagem seja testado antes de qualquer afirmação sobre ele.

7.1 Delineamento

Propomos um experimento randomizado em grupos paralelos [Wohlin *et al*., 2012], em uma disciplina de desenvolvimento de software em que o uso de agentes de IA é permitido. Há duas condições. Na condição *kata*, o estudante usa o agente com o ciclo, em versão reduzida: planejamento antes do código (FIT e THINK, caso 1 da Seção 6), declaração e justificativa dos arquivos alterados (SURGICAL), verificação (VERIFY, com `kata --check-only`) e revisão antes da entrega (JUDGE, com `kata --judge`, caso 2). Na condição *controle*, usa o mesmo agente, com o corretor habitual da disciplina e sem nenhuma etapa do kata. A atribuição é feita por sorteio, individual ou por equipe quando o trabalho é em equipe, estratificado por turma, com a sequência gerada por quem não avalia a turma e ocultada até o momento da alocação, e o mesmo docente conduz as duas condições. Se a disciplina não puder sortear, o estudo passa a ser um quase-experimento, e a seleção entra como ameaça à H2. Assim, nenhuma turma nem docente fica em uma só condição, o que controla o efeito de turma e de docente, ao custo de aumentar o risco de contaminação entre colegas (Seção 7.4). O kata não tem comando que dispense fases individuais (caso 7), então a versão reduzida é uma instrução da disciplina, e não um modo da ferramenta. As fases não selecionadas (SIMPLIFY, INTENT, TWIN CHECK, ARTIFACT e REPORT) são respondidas com o padrão ou deixadas de lado, ficam fora das medidas, e o docente confere o cumprimento nos arquivos de tarefa. O JUDGE exige que o arquivo de tarefa traga os blocos `surgical` e `verify` preenchidos.

Descartamos o delineamento cruzado como desenho principal, por dois motivos. O primeiro é que a H3 compara a retenção entre quem usou o kata e quem não usou, e no cruzado todas as equipes o usariam em algum período. O segundo é que uma intervenção pedagógica deixa efeito residual: o que se aprende com o kata em um período contamina o período seguinte em que a pessoa faz o papel de controle.

Um cruzado só seria aceitável como análise secundária da H1, com a ordem como fator e esse limite declarado.

7.2 Hipóteses e Medidas

As quatro hipóteses e suas medidas são definidas antes da coleta. A H2 é a hipótese principal, e as demais são secundárias.

- **H1: declaração do raciocínio.** Quem usa o kata declara premissas mais verificáveis e justifica mais arquivos. Para que se compare o conteúdo, e não a presença do campo, a condição controle preenche um formulário com os mesmos campos (problema, premissas, critério de pronto e arquivos alterados com a justificativa de cada um) sem o kata. O formulário do controle é preenchido *depois* da implementação e antes da entrega, para que a diferença entre as condições seja planejar antes do código e não ter o registro. Duas pessoas avaliadoras, sem saber a condição, aplicam uma rubrica de cinco dimensões, pontuadas de 0 a 2 com âncoras escritas. As dimensões são premissa explícita, premissa verificável (que um comando, um arquivo ou um teste pode conferir), alternativa considerada, critério de pronto observável e justificativa de cada arquivo. As avaliadoras são treinadas em 10 registros do estudo-piloto, e a concordância, medida pelo *kappa* de Cohen [Cohen, 1960], deve atingir um valor-alvo fixado antes da coleta (por exemplo, 0,7); abaixo disso, a rubrica é revista e o treino repetido.
- **H2 (principal): integridade da entrega.** Há menos entregas cujos testes visíveis passam e cujos testes ocultos falham. O mecanismo esperado é duplo: o critério de pronto escrito antes do código (só na condição kata, pois o controle registra o seu depois da implementação) orienta que testes escrever, e o JUDGE devolve à pessoa, antes da entrega, o teste enfraquecido, a falsa conclusão e o escopo excedente, o que a condição controle não recebe. Como a intervenção é o conjunto, o estudo não separa o efeito do planejamento do efeito do JUDGE. O JUDGE faz parte da intervenção e seu veredito não entra na medida, para a hipótese não ser circular. Os testes ocultos são escritos antes da coleta por quem não conduz a disciplina, e uma segunda pessoa os valida contra o requisito. As proporções supostas na Tabela 8 são calibradas no estudo-piloto.
- **H3: retenção.** Todas as pessoas fazem uma tarefa sem agente depois de um intervalo fixado antes da coleta (por exemplo, uma semana após a última atividade, como no pós-teste de Kazemitabaar *et al*. [2023]), e quem usou o kata não tem desempenho inferior ao de quem não usou, o que indicaria que o andaime não substituiu a aprendizagem. A margem de não inferioridade, por exemplo meio desvio-padrão, é fixada antes da coleta.
- **H4: custo.** O ciclo aumenta o tempo por tarefa. O tempo é registrado nas duas condições para dimensionar o custo, que este artigo não mede com estudantes.

7.3 Amostra e Análise

O tamanho da amostra é calculado por hipótese, com poder de 0,80. A H2, por ser a principal, é testada com nível de significância de 0,05. As secundárias (H1, H3 e H4) só são interpretadas se a H2 for significativa (teste hierárquico), o que mantém o erro familiar em 0,05, e são testadas com a correção de Holm entre as três, cujo pior caso exige nível de 0,0167 para cada uma. A Tabela 8 mostra os números. Os efeitos são suposições de planejamento, e não resultados: nenhum dado deste artigo os sustenta, e devem ser revistos com um estudo-piloto.

A H2 determina o plano: cerca de 121 estudantes por grupo, ou 142 com 15% de perda. Se a atribuição for por equipe, o agrupamento pede o efeito de delineamento: com equipes de três e correlação intraclasse suposta de 0,2, o fator é 1,4, e o plano sobe para cerca de 169 por grupo sem perda e 199 com 15% de perda. A análise usa regressão logística (H2) e modelos lineares (H1, H3 e H4), com turma como efeito aleatório e, quando houver equipes, equipe também. Relata tamanhos de efeito com intervalos de confiança de 95% e segue a intenção de tratar, registrando o uso de agentes fora da condição prevista. A perda é registrada por condição e por motivo. Como a condição kata dá mais trabalho e pode perder mais gente, uma análise de sensibilidade refaz a estimativa com perda de 15% no controle e de 25% no kata, o que pediria cerca de 162 estudantes no grupo kata (121 dividido por 0,75) e 142 no controle (121 dividido por 0,85), ou seja, cerca de 304 no total contra 284. Os 142 por grupo valem só se a perda for igual nas duas condições.

7.4 Ameaças e Mitigação

As ameaças previsíveis, na taxonomia de Wohlin *et al*. [2012], são as seguintes. A contaminação entre colegas da mesma turma é a maior, porque a atribuição é individual: mitiga-se entregando o kata só à condição correspondente, pedindo que não compartilhem a ferramenta e registrando conversas entre as condições. O efeito do docente é controlado por ele conduzir as duas condições, e passa a ser o risco de tratar os grupos de modo diferente, o que se mitiga com roteiro único de atendimento. O uso de IA fora da condição é mitigado pela declaração obrigatória a cada entrega. A deriva do agente é outra ameaça, porque o desfecho depende do agente hospedeiro e do modelo, que mudam entre semestres. Mitiga-se usando o mesmo agente, o mesmo modelo e a mesma versão nas duas condições, registrando-os a cada entrega. Uma troca de versão no meio do estudo conta como desvio de protocolo. A diferença de tempo entre as condições é mitigada pelo registro de tempo (H4) e pela mesma carga de trabalho prevista. A cegueira incompleta das pessoas avaliadoras, porque só o kata deixa um arquivo de tarefa, é mitigada pela avaliação do formulário da condição controle sem identificação da condição. A perda diferencial é outra ameaça, porque quem desiste da condição kata tende a ser quem teve mais dificuldade, o que favorece o grupo e é tratado pelo registro de perda por condição e pela análise de sensibilidade da Seção 7. Por fim, o JUDGE tem desvios conhecidos (Tabela 1): um estudante que controla o repositório pode escapar dele. Isso enfraquece o mecanismo da H2 sem invalidar a medida, porque o desfecho vem dos testes ocultos, e não do veredito.

7.5 Ética e Pré-registro

Por envolver seres humanos, o estudo exigiria aprovação ética conforme a legislação vigente [Brasil, 2024; Conselho

Nacional de Saúde, 2016] e consentimento específico para o uso dos registros. Os dados seriam tratados conforme a LGPD [Brasil, 2018], com pseudonimização dos arquivos de tarefa e do histórico do Git. A participação não pode afetar a nota, e o veredito do JUDGE não entra na avaliação da disciplina. Por causa da relação de poder entre docente e estudante, o recrutamento e o consentimento são conduzidos por quem não avalia a turma, e a recusa e a desistência não trazem prejuízo. As avaliadoras da H1 recebem registros pseudonimizados, e o docente só vê resultados agregados depois de fechadas as notas. As hipóteses, as medidas, as margens e o plano de análise devem ser pré-registrados em plataforma aberta antes da coleta.

8 Discussão

Esta seção reúne as implicações da proposta para quem ensina, para quem desenvolve e para quem pesquisa. Todas são condicionais, porque o artigo não traz dados de estudantes.

Docentes. (i) O revisor independente do kata poderia ser uma etapa automática de entrega, ao lado do corretor de exercícios, desde que o veredito seja lido como indício (Seção 3.4). (ii) O ciclo completo deve ser planejado como atividade de aprendizagem, com tempo previsto. (iii) A auditoria e a Tabela 6 oferecem um ponto de partida para o retorno sobre o processo, que ainda precisa ser testado com turmas.

Equipes de desenvolvimento. (i) O `kata --check-only` e o `kata --judge` podem rodar em integração contínua, e `--trusted-base` fecha, no servidor, parte dos desvios da Tabela 1. (ii) Em repositórios com falhas preexistentes, a auditoria cega ao contexto pode acusar falsamente, e declarar a base conhecida evitaria isso. (iii) O artigo não oferece evidência de que o ciclo melhore o resultado.

Pesquisa. (i) Medir o efeito exige desfechos independentes do detector da própria ferramenta, como testes ocultos e avaliação cega. (ii) O custo de tempo deve ser reportado ao lado do desfecho. (iii) Hipóteses, medidas e margens devem ser pré-registradas.

O que o artigo não responde. Não sabemos se o kata melhora a aprendizagem, se a retenção sem agente é preservada, quanto tempo o ciclo custa a estudantes, se docentes leem os registros, se a rubrica da Tabela 6 funciona em sala nem como a ferramenta se comporta em disciplinas de infraestrutura, dados, pesquisa ou documentação, onde só a parte executável declarada pelo projeto é verificada. O protocolo da Seção 7 trata da aprendizagem, da retenção e do tempo, e as demais perguntas pedem estudos próprios.

Condições para uma primeira adoção. Para quem quiser experimentar a proposta antes de qualquer estudo, sugerimos um piloto pequeno. Ele usaria uma turma, os casos 1 e 2, tarefas curtas e o ciclo reduzido (planejamento, justificativa dos arquivos, verificação e JUDGE), com um docente que leia os registros e uma conversa com a turma sobre o que a ferramenta mede. O piloto serve para calibrar o tempo, a rubrica e a aceitação, e não para afirmar efeito.

9 Limitações da Pesquisa

Este é um artigo de proposta, e por isso as limitações abaixo só se corrigem com estudos com pessoas e com avaliações independentes da ferramenta. Organizamo-las pela taxonomia de Wohlin *et al*. [2012].

Validade interna. Nós mesmos construímos o kata, escolhemos os casos de uso e reproduzimos os desvios do verificador, e o rascunho do texto foi feito com um assistente de IA (declarado ao final). As oito execuções (Seção 3.5 e Apêndice A) são nossas, feitas em projetos pequenos, com respostas e arquivos escritos por nós, e não representam o que estudantes fariam nem medem desempenho. O comportamento do verificador descrito na Seção 3 não tem, neste artigo, evidência independente, e o leitor só pode reexecutar as oito execuções de demonstração.

Validade externa. Não há evidência com estudantes, docentes ou outras turmas, e os casos de uso são hipóteses sem teste em sala. A verificação técnica da ferramenta, relatada à parte, cobriu código Python. Outras linguagens e os adaptadores de domínio não foram avaliados.

Validade de constructo. A correspondência da Tabela 5 é uma interpretação, e nenhum instrumento mediu se as fases evocam os processos cognitivos que lhes atribuímos. As formas de fraude que o JUDGE detecta (teste enfraquecido, escopo não declarado, falsa conclusão) são indicadores de integridade do trabalho, e não de aprendizagem. O kata verifica a presença e a coerência dos registros, e não a qualidade do julgamento.

Validade de conclusão. Este artigo não faz análise estatística. As hipóteses H1 a H4 e o tamanho de amostra são um plano, e os números da Tabela 8 partem de efeitos supostos, e não de efeitos observados.

Versão e verificador. O texto descreve a versão 0.10.0 (tag `v0.10.0`), publicada em outubro de 2026; as diferenças em relação à 0.7.2 estão indicadas na Seção 3 e em Materiais. Os desvios da Tabela 1 mostram que o veredito não pode ser a única base de uma decisão sobre um estudante.

10 Conclusão e Trabalhos Futuros

Este artigo propôs o kata, um ciclo de nove fases cuja parte objetiva é código Python testado, como ferramenta de apoio ao ensino de desenvolvimento de software com agentes de IA. A contribuição é a proposta: a ferramenta e seus limites conhecidos, uma fundamentação pedagógica, dois casos de uso para o primeiro teste, seis extensões e um protocolo de estudo com estudantes. O artigo não traz evidência sobre aprendizagem nem sobre o custo de tempo para estudantes. Quanto às questões, a fundamentação pedagógica relaciona cada fase a um processo cognitivo e a uma evidência registrada (QP1). Os casos de uso indicam fluxo, evidência e ressalva, e os desvios reproduzidos do verificador impedem usar o veredito como prova única de integridade (QP2). O protocolo propõe, em grupos paralelos, quatro hipóteses com uma principal, uma amostra de cerca de 142 estudantes por grupo calculada por hipótese e uma análise pré-registrada (QP3). O registro por fase e a auditoria fornecem retorno sobre o processo, mas a ferramenta só verifica a presença do raciocínio, nunca sua qualidade.

As implicações para a prática são condicionais. Docentes que queiram experimentar a proposta podem usar o revisor independente como verificação automática de entregas, lendo o veredito como indício, e tratar o ciclo como atividade

de aprendizagem. Os casos mais indicados para um primeiro teste em sala são o planejamento antes do código e o ensino de testes (casos 1 e 2), que o protocolo cobre. Os demais ficam como extensão.

Os trabalhos futuros são três: executar o protocolo da Seção 7 com estudantes, começando pelos casos 1 e 2; fechar os desvios do verificador, com uma opção de falha fechada para `UNVERIFIABLE`; e declarar as falhas preexistentes da base para que a auditoria não puna dívida herdada.

Declarações complementares

Contribuições dos autores

W.A.N. concebeu e implementou o kata, projetou a proposta e o protocolo, produziu a execução de demonstração, revisou, editou e aprovou o manuscrito submetido, cujo rascunho foi produzido com IA generativa conforme a declaração de uso de IA (Taxonomia CRediT: conceituação, software, investigação, redação – revisão e edição).

Financiamento

Omitido para revisão.

Conflitos de interesse

A autoria declara o seguinte conflito de interesses: é a desenvolvedora do kata, e a descrição e os limites da ferramenta são autorrelatados.

Disponibilidade de dados e materiais

O software e os materiais deste artigo serão disponibilizados após o aceite.

Declaração de utilização de inteligência artificial (IA)

Em atendimento ao Código de Conduta para autores da SBC [2024], declaramos o uso de IA generativa de duas formas. *Redação*: o rascunho deste manuscrito foi redigido pelo Claude, da Anthropic, operado pelo Claude Code, a partir do código, da documentação e de um manuscrito anterior da autoria sobre o kata (endereçado a periódico de engenharia de software), e depois revisado pela autoria; a estrutura, o enquadramento educacional e o reposicionamento da proposta foram sugeridos pelo assistente e submetidos à decisão da autoria. *Software*: o próprio kata e seus testes foram desenvolvidos com assistência de modelos Claude, e o histórico de *commits* registra o modelo coautor de cada alteração. A Figura 1 foi escrita em TikZ (pacote gráfico do LaTeX) pela autoria e pelo assistente. A autoria é responsável pelo conteúdo, e as referências de Educação em Computação tiveram seus metadados conferidos no Crossref.

Outras informações relevantes

Sobreposição com outro manuscrito. A verificação técnica do kata (cenários adversariais e sessões de agentes em repositórios públicos) foi relatada em manuscrito separado da autoria, [omitido para revisão]. Este artigo não reproduz esses dados. A descrição do ciclo, do arquivo de tarefa, do verificador e dos desvios sobrepõe-se em parte à descrição da ferramenta naquele manuscrito, e a fundamentação pedagógica (Seção 5), os casos de uso (Seção 6) e o protocolo de estudo (Seção 7) não constam nele.

Participantes. Este artigo não envolveu participantes humanos. O protocolo da Seção 7 exigirá aprovação ética antes de qualquer coleta.

Referências

Almeida, A., Guerrero, D. D. S., & Andrade, W. L. (2025). Avaliando o efeito da criação de testes na aprendizagem e no desempenho de estudantes de programação. In *Anais do XXXVI Simpósio Brasileiro de Informática na Educação (SBIE 2025)*, pages 140–152. DOI: 10.5753/sbie.2025.12058.

Anderson, L. W. & Krathwohl, D. R. (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, New York.

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525. DOI: 10.1109/TSE.2014.2372785.

Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (2023). Programming is hard - or at least it used to be: Educational opportunities and challenges of AI code generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, pages 500–506. DOI: 10.1145/3545945.3569759.

Bedin, H. A. T., Marioti, M. B., & Jaques, P. (2025). Generative AI assisting programming learning: Functional roles, comparative analysis, and emerging challenges. In *Anais do XXXVI Simpósio Brasileiro de Informática na Educação (SBIE 2025)*, pages 409–426. DOI: 10.5753/sbie.2025.12361.

Biggs, J. (1996). Enhancing teaching through constructive alignment. *Higher Education*, 32(3):347–364. DOI: 10.1007/bf00138871.

Bispo Jr., E. L., Raabe, A., Matos, E., Maschio, E., Barbosa, E. F., Carvalho, L. G., Bittencourt, R. A., Duran, R. S., & Falcão, T. P. (2020). Tecnologias na educação em computação: Primeiros referenciais. *Revista Brasileira de Informática na Educação*, 28:509–527. DOI: 10.5753/rbie.2020.28.0.509.

Black, P. & Wiliam, D. (1998). Assessment and classroom learning. *Assessment in Education: Principles, Policy & Practice*, 5(1):7–74. DOI: 10.1080/0969595980050102.

Brasil (2018). Lei n.º 13.709, de 14 de agosto de 2018 — lei geral de proteção de dados pessoais (LGPD). Diário Oficial da União, `https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm`.

Brasil (2024). Lei n.º 14.874, de 28 de maio de 2024: dispõe sobre a pesquisa com seres humanos e institui o sistema nacional de Ética em pesquisa com seres humanos. Diário Oficial da União, `https://www.planalto.gov.br/ccivil_03/_ato2023-2026/2024/lei/l14874.htm`.

Camilo, H. T. & Hauck, J. C. R. (2026). Aplicações de LLMs no ensino de programação orientada a objetos: Um mapeamento sistemático da literatura. In *Anais do XXXIV Workshop sobre Educação em Computação (WEI 2026)*, pages 36–49. DOI: 10.5753/wei.2026.20394.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46. DOI: 10.1177/001316446002000104.

Conselho Nacional de Saúde (2016). Resolução nº 510, de 7 de abril de 2016. https://www.gov.br/conselho-nacional-de-saude/pt-br/atos-normativos/resolucoes/2016/resolucao-no-510.pdf.

da Silva, E. P., Caceffo, R. E., & Azevedo, R. (2023). Passar nos casos de teste é suficiente? identificação e análise de problemas de compreensão em códigos corretos. In *Anais do III Simpósio Brasileiro de Educação em Computação (EDUCOMP 2023)*, pages 119–129. DOI: 10.5753/educomp.2023.228346.

da Silva, T. L., Vidotto, K. N. S., Tarouco, L. M. R., & da Silva, P. F. (2024). Potencialidades do uso de inteligência artificial generativa como apoio ao ensino de programação. In *Anais do XXXV Simpósio Brasileiro de Informática na Educação (SBIE 2024)*, pages 1942–1956. DOI: 10.5753/sbie.2024.242711.

Daun, M. & Brings, J. (2023). How ChatGPT will change software engineering education. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, pages 110–116. DOI: 10.1145/3587102.3588815.

Delgado, C. A. D. M., Galvão de Carvalho, L. S., & Fortes, R. S. (2026). Uso de juízes online no ensino de programação: desafios e oportunidades. In *Anais Estendidos do VI Simpósio Brasileiro de Educação em Computação (EDUCOMP 2026)*, pages 139–143. Resumo de mesa temática. DOI: 10.5753/educomp_estendido.2026.19666.

Denny, P., Prather, J., Becker, B. A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B. N., Santos, E. A., & Sarsa, S. (2024a). Computing education in the era of generative AI. *Communications of the ACM*, 67(2):56–67. DOI: 10.1145/3624720.

Denny, P., Smith, D. H., Fowler, M., Prather, J., Becker, B. A., & Leinonen, J. (2024b). Explaining code with a purpose: An integrated approach for developing code comprehension and prompting skills. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, pages 283–289. DOI: 10.1145/3649217.3653587.

Edwards, S. H. (2004). Using software testing to move students from trial-and-error to reflection-in-action. In *Proceedings of the 35th SIGCSE technical symposium on Computer science education*, pages 26–30. DOI: 10.1145/971300.971312.

Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022). The robots are coming: Exploring the implications of OpenAI Codex on introductory programming. In *Proceedings of the 24th Australasian Computing Education Conference*, pages 10–19. ACM. DOI: 10.1145/3511861.3511863.

forrestchang (2026). andrej-karpathy-skills: A single CLAUDE.md to improve Claude Code behavior, derived from Andrej Karpathy's observations on LLM coding pitfalls. Repositório GitHub: https://github.com/forrestchang/andrej-karpathy-skills. Licença MIT; baseado em publicação de Karpathy no X: https://x.com/karpathy/status/2015883857489522876; sem revisão por pares; acesso em 3 out. 2026.

Garousi, V., Giray, G., Tüzün, E., Catal, C., & Felderer, M. (2019). Aligning software engineering education with industrial needs: A meta-analysis. *Journal of Systems and Software*, 156:65–83. DOI: 10.1016/j.jss.2019.06.044.

Hattie, J. & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1):81–112. DOI: 10.3102/003465430298487.

Hellas, A., Leinonen, J., Sarsa, S., Koutcheme, C., Kujanpää, L., & Sorva, J. (2023). Exploring the responses of large language models to beginner programmers' help requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*, pages 93–105. DOI: 10.1145/3568813.3600139.

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*. arXiv:2310.01798; https://arxiv.org/abs/2310.01798.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*. arXiv:2310.06770; https://arxiv.org/abs/2310.06770.

Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2024). AI agents that matter. arXiv:2407.01502; https://arxiv.org/abs/2407.01502. Preprint.

Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pages 1–23. DOI: 10.1145/3544548.3580919.

Keuning, H., Jeuring, J., & Heeren, B. (2018). A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1):1–43. DOI: 10.1145/3231711.

Kratzke, N. (2019). Smart like a fox: How clever students trick dumb automated programming assignment assessment systems. In *Proceedings of the 11th International Conference on Computer Supported Education*, pages 15–26. DOI: 10.5220/0007424800150026.

Lau, S. & Guo, P. (2023). From "ban it till we understand it"to "resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*, pages 106–121. DOI: 10.1145/3568813.3600138.

Manheim, D. & Garrabrant, S. (2019). Categorizing variants of Goodhart's law. arXiv:1803.04585; https://arxiv.org/abs/1803.04585. Preprint.

Oliveira, A. A. & Teixeira, M. M. (2026). Proposta de um grau de uso da inteligência artificial no contexto educacional: Critérios éticos e pedagógicos para orientar a prática docente. In *Anais do VI Simpósio Brasileiro de Educação em Computação (EDUCOMP 2026)*, pages 214–227. DOI: 10.5753/educomp.2026.18562.

Pimentel, E. P., Lima da Silva, L. D., Takeuti, R. H., & Braga, J. C. (2025). Abordagem com LLM para geração automatizada de feedback no ensino de programação de computadores. In *Anais do XXXVI Simpósio Brasileiro de Informática na Educação (SBIE 2025)*, pages 591–603. DOI: 10.5753/sbie.2025.12543.

Prather, J., Denny, P., Leinonen, J., Becker, B. A., Albluwi, I., Craig, M., Keuning, H., Kiesler, N., Kohn, T., Luxton-Reilly, A., MacNeil, S., Petersen, A., Pettit, R., Reeves, B. N., & Savelka, J. (2023a). The robots are here: Navigating the generative AI revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, pages 108–159. ACM. DOI: 10.1145/3623762.3633499.

Prather, J., Reeves, B. N., Denny, P., Becker, B. A., Leinonen, J., Luxton-Reilly, A., Powell, G., Finnie-Ansley, J., & Santos, E. A. (2023b). "it's weird that it knows what i want": Usability and interactions with Copilot for novice programmers. *ACM Transactions on Computer-Human Interaction*, 31(1):1–31. DOI: 10.1145/3617367.

Prather, J., Reeves, B. N., Leinonen, J., MacNeil, S., Randrianasolo, A. S., Becker, B. A., Kimmel, B., Wright, J., & Briggs, B. (2024). The widening gap: The benefits and harms of generative AI for novice programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research*, volume 1, pages 469–486. ACM. DOI: 10.1145/3632620.3671116.

Sahir619 (2026). The Fable method: The Fable workflow. Repositório GitHub, commit 88b5cf3: https://github.com/Sahir619/fable-method. Sem revisão por pares; acesso em 11 set. 2026.

SBC (2024). Código de conduta para autores em publicações da SBC. SBC OpenLib (SOL): https://sol.sbc.org.br/index.php/indice/conduta. Sociedade Brasileira de Computação.

Simão de Deus, W., Marcolino, A. S., Avellar, G. M. N., Oliveira, K. K. S., & Barbosa, E. F. (2024). Avaliando resoluções de exercícios introdutórios de programação na era das IAs generativas: Um estudo de caso com o ChatGPT. In *Anais do XXXV Simpósio Brasileiro de Informática na Educação (SBIE 2024)*, pages 1645–1657. DOI: 10.5753/sbie.2024.242283.

Spacco, J. & Pugh, W. (2006). Helping students appreciate test-driven development (tdd). In *Companion to the 21st ACM SIGPLAN symposium on Object-oriented programming systems, languages, and applications*, pages 907–913. DOI: 10.1145/1176617.1176743.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2 edition.

Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2):89–100. DOI: 10.1111/j.1469-7610.1976.tb00381.x.

Zhao, B., Srikanth, D., Wu, Y., & Jiang, Z. (2026). SpecBench: Measuring reward hacking in long-horizon coding agents. arXiv:2605.21384; https://arxiv.org/abs/2605.21384. Preprint, sem revisão por pares.

```
$ kata --judge --task dividir-por-zero
  VEREDITO: REFUTED
    [high] spec_betrayal
      intenção não alinhada: código, teste e spec discordam
      e nenhuma resolução foi registrada
  Re-execução: lint [ok]  teste [ok]  coverage [ok]
  (código de saída 1)

# conflict_resolution preenchido com uma frase
$ kata --judge --task dividir-por-zero
  VEREDITO: VERIFIED
  Claims aceitas sem verificação (não re-executáveis):
    - critério de sucesso satisfeito
    - conflito código/teste/spec resolvido por declaração:
      o teste prevalece sobre o código atual (...)
  (código de saída 0)
```

Listagem 4. Execução 2: o JUDGE sobre a tarefa da Execução 1, com o conflito de INTENT sem resolução e depois com uma resolução de uma frase. Saída real, resumida e com ícones trocados por marcadores ASCII.

```
$ kata --judge --task dividir-por-zero
  VEREDITO: REFUTED
    [high] weakened_checks
      tests/test_calculadora.py: corpo vazio (só pass)
    [high] false_completion
      lint re-executado falhou, mas o relatório
      afirma que passou
  Re-execução: lint [x]  teste [ok]  coverage [ok]
  (código de saída 1)

$ kata --check-only
  > ruff check src/ tests/   [x] F401 `pytest` imported
                                 but unused
  > pytest tests/            [ok] passou
  > coverage (gate >= 70%)   [ok] passou (75.0%)
  [x] KATA CYCLE - REJEITADO  (código de saída 1)
```

Listagem 5. Execução 3: o corpo de `test_dividir_por_zero` vira `pass`. Saída real, resumida.

Apêndice A: Execuções Reais Adicionais do Kata

Este apêndice reúne as Execuções 2 a 8 do kata, que complementam a Execução 1 da Seção 3.5. As saídas correspondem à versão 0.10.0, com a qual as saídas de `demos/saidas/` foram regravadas, exceto a 01c, que exige um interpretador sem `ruff` e `pytest`. Em relação à 0.7.2, o relatório passou a usar os papéis de verificação (lint, teste, cobertura) e a imprimir a evidência da execução; a Execução 5 mede 26,1% (6 de 23 *flags*) porque a 0.8.0 a 0.10.0 acrescentaram seis *flags* ao CLI. As Listagens 4 a 6 e 7 resumem as sete execuções reais adicionais, todas feitas por nós, em projetos pequenos e descartáveis, com respostas e arquivos escritos por nós mesmos. O script `demos/reproduzir.sh` reconstrói os projetos, inclusive o da Execução 1 (Seção 3.5), e regrava as saídas completas em `demos/saidas/`. Os projetos não têm *commits*, e o JUDGE leu as mudanças no índice do Git. A Tabela 9 resume o que cada uma mostra. Elas ilustram o comportamento da ferramenta e não medem desempenho. As Execuções 1 a 4 usam o domínio de programação, e as Execuções 5 a 8 usam, na ordem, os de documentação, análise de dados, infraestrutura e pesquisa, de modo que os cinco domínios têm ao menos uma execução.

A Execução 2 (Listagem 4) mostra os dois lados da fase INTENT. Com o desacordo sem resolução, o JUDGE refuta a tarefa por traição da especificação, embora lint, testes e cobertura passem. Basta preencher `conflict_resolution`

```
$ kata --audit --task entrega-aluno
  [x] FIT: faked
      rota e trivialidade não classificadas por humano
  [x] THINK: faked
      assumptions nunca declaradas
  [x] INTENT: faked   [x] SIMPLIFY: faked
  [x] SURGICAL: faked [x] VERIFY: faked
      sucesso afirmado sem evidência de execução
  [x] TWINS: faked
  Audit encontrou 7 fake(s) e 0 skip(s).
  (código de saída 1)
```

Listagem 6. Execução 4: auditoria de um arquivo de tarefa escrito à mão, com todas as fases marcadas como respondidas e vazias. Saída real, resumida.

```
# 5 docs: README cita --auditar, que não existe
  > verifica_doc.py test   [x] flags citadas que não
                               existem: ['--auditar']
  [x] KATA CYCLE - REJEITADO   (código de saída 1)
  # README corrigido
  > coverage (gate >= 25%) [ok] passou (26.1%)
  [ok] KATA CYCLE - APROVADO   (código de saída 0)

# 6 data-analysis: análise remove 2 de 10 linhas
  > verifica_dados.py test [x] linhas removidas sem
                               registro: 2 de 10
  [x] KATA CYCLE - REJEITADO   (código de saída 1)
  # análise v2 registra a remoção em resultado.txt
  > coverage (gate >= 70%) [ok] passou (80.0%)
  [ok] KATA CYCLE - APROVADO   (código de saída 0)

# 7 devops: nginx:latest e healthcheck desativado
  > verifica_infra.py test [x] web: imagem sem versão
        fixa (nginx:latest); web: healthcheck ausente
        ou desativado
  [x] KATA CYCLE - REJEITADO   (código de saída 1)
  # nginx:1.27.2 e healthcheck com curl
  > coverage (gate >= 100%) [ok] passou (100.0%)
  [ok] KATA CYCLE - APROVADO   (código de saída 0)

# 8 research: DOI inexistente e afirmação sem fonte
  > verifica_fontes.py test [x] DOI não resolve:
        10.1145/9999999.9999999; afirmação sem fonte:
        A maioria dos cursos já ensina verificação (...)
  [x] KATA CYCLE - REJEITADO   (código de saída 1)
  # DOI e título reais; afirmação sem fonte trocada
  > coverage (gate >= 100%) [ok] passou (100.0%)
  [ok] KATA CYCLE - APROVADO   (código de saída 0)
```

Listagem 7. Execuções 5 a 8: verificações declaradas pelo projeto em `.kata/config.yaml` nos domínios `docs`, `data-analysis`, `devops` e `research`. Cada bloco mostra a rejeição e a aprovação depois da correção. Os comandos declarados e as saídas completas estão em `demos/`. Saída real, resumida.

com uma frase para o veredito virar `VERIFIED`, e a frase aparece na lista de afirmações aceitas sem verificação. O kata, portanto, obriga o registro do raciocínio e não confere a sua qualidade, que é o limite que a Seção 5 atribui à ferramenta.

A Execução 3 (Listagem 5) mostra o caso que um corretor só por testes deixaria passar. Com o corpo do teste trocado por `pass`, o `pytest` passa e a cobertura continua em 75%, e só o JUDGE acusa o enfraquecimento. A segunda acusação, de falsa conclusão, vem de um efeito colateral: o teste esvaziado deixou de usar `pytest`, e o `ruff` acusou a importação sem uso. Reexecutamos a variante sem a importação (`demos/saidas/03c-judge-weakened-sem-import.txt`): o lint passa e o JUDGE acusa só o enfraquecimento.

A Execução 4 (Listagem 6) mostra a auditoria sobre um arquivo escrito à mão. Todas as sete fases saem como *falsificadas*, com o risco de cada uma, e o código de saída é 1. O arquivo, porém, seria graduado como seguido se cada campo tivesse uma frase qualquer, e isso é o que a Execução 1 mostra no INTENT.

A Execução 5 (Listagem 7) mostra o domínio de documentação, que não tem verificação própria em Python. O projeto declara três comandos em `.kata/config.yaml`, que o kata roda como estão. O teste reprova a citação da *flag* `--auditar`, que não existe, e a "cobertura" vira a fração das *flags* do kata que o README documenta (26,1%, ou 6 de 23), lida pelo `coverage_pattern`. O denominador subiu de 17 para 23 porque a 0.8.0 acrescentou `--stamp` e `--commit`, a 0.9.0 acrescentou `--verify-task` e o par `--criterion-met`/`--no-criterion-met` e a 0.10.0 acrescentou `--artifact-task`. O portão de 25% é arbitrário, e o exemplo só mostra que a mecânica do kata serve a documentos.

A Execução 6 (Listagem 7) mostra o domínio de análise de dados e a fraude de remoção silenciosa de linhas. O adaptador manda o agente registrar o critério de qualquer remoção, e isso é só instrução. O projeto a torna executável: o teste declarado reprova a análise que descarta 2 de 10 linhas sem registro e passa quando o registro existe, e a "cobertura" vira a fração de linhas usadas (80,0%). O portão de 70% é arbitrário, e a verificação existe porque o projeto a escreveu. O kata não a produz sozinho.

A Execução 7 (Listagem 7) mostra o domínio de infraestrutura. O `docker compose config` sozinho aceitaria a configuração inicial, porque a sintaxe está correta. O que reprova é um *script* do projeto que implementa duas regras do adaptador, a imagem com versão fixa e o *healthcheck* ativo, e a "cobertura" vira a fração de serviços com *healthcheck* (100,0% depois da correção). O Docker foi usado só para validar a sintaxe, sem subir contêiner.

A Execução 8 (Listagem 7) mostra o domínio de pesquisa e a fonte inventada, o problema que o adaptador chama de `fabricated_source`. O *script* do projeto consulta o Crossref e reprova um DOI que não resolve e uma afirmação sem fonte. A verificação cobre só fontes com DOI e só confere o título, e não que a fonte sustente a afirmação, que continua sendo trabalho de quem lê. Ela também exige rede.

Apêndice B: Protocolo da Busca Sistemática Curta

Para a produção nacional, fizemos uma busca sistemática curta em 3 de outubro de 2026 no SOL, que indexa o SBIE, o WEI e o EDUCOMP (inclusive os anais estendidos), e na RBIE. Aplicamos oito *strings* em português ao período de 2022 a 2026: IA generativa e programação; ChatGPT e programação; juiz online; correção automática e programação; ensino de teste; criação de testes; integridade acadêmica e programação; e engenharia de software com IA generativa e ensino. Como o site da RBIE não retornou resultado nem para "programação", consultamos a RBIE pelo Crossref (ISSN, período e as mesmas *strings*, 25 registros por *string*, com filtro de palavra no título). Obtivemos 126 registros, 75 distintos e 52 no período. Incluímos os trabalhos que tratam de (a) IA generativa no ensino de programação ou de engenharia de software, (b) correção automática, retorno e juízes online, (c) ensino de teste de software ou (d) inte-

gridade e uso aceitável de IA. Excluímos os de outro tema. A triagem foi feita por título e resumo por uma única pessoa, com apoio de um assistente de IA, sem concordância entre avaliadores: incluímos 27 e excluímos 25. Os arquivos `busca/busca_sol.py`, `busca/resultados_busca.csv`, `busca/triagem.csv` e `busca/contagens.txt` permitem refazer a busca e conferir as contagens.

Tabela 2. Campos do arquivo de tarefa das Listagens 2 e 3: fase, quem preenche e o que o kata faz com o bloco. A auditoria gradua um bloco como *falsificado* quando falta o campo-chave indicado ou, em `verify`, quando há sucesso afirmado sem nenhuma evidência. Fonte: leitura do código do kata 0.8.0 (`src/kata/cli.py`, `judge.py` e `report.py`).

Bloco	Fase	Quem preenche	O que o kata faz com o bloco
`task`, `status`, `domain`	Geral	O kata	Identifica a tarefa e o domínio; `status` vira `approved` ou `rejected` ao fim do VERIFY
`kata_version`	Geral	O kata, no VERIFY	Versão que produziu a evidência; o JUDGE a compara com a que está rodando e ressalva a diferença
`done`	THINK, VERIFY	A pessoa	Critério de pronto declarado antes do código e confrontado com o resultado no VERIFY
`fit`	FIT	A pessoa	Registra rota e justificativa; `trivial` parte da medida do *diff* e exige confirmação da pessoa. Auditoria: campo-chave `reason`
`think`	THINK	A pessoa	Problema, premissas, alternativas e incógnitas. Auditoria: campo-chave `problem`
`simplify`	SIMPLIFY	A pessoa	Três confirmações de minimalidade. Auditoria: campo-chave `minimum_code`
`intent`	INTENT	A pessoa	Código, teste e especificação. Com `all_agree: false` e sem `conflict_resolution`, o JUDGE aponta traição da especificação (alta gravidade); a resolução é autodeclarada e aceita sem verificação. Auditoria: `code_does`
`surgical`	SURGICAL	A pessoa	Lista de arquivos e justificativas. O JUDGE compara a lista com o *diff* real (escopo excedente). Auditoria: `files`
`verify`	VERIFY	O kata (e a pessoa, em `success_criteria_met`)	Resultados de lint, tipos (quando declarados), testes e cobertura vêm de códigos de saída; `gate_used`, `commands` e `ran_at` registram a evidência da execução. O JUDGE reexecuta e aponta falsa conclusão se divergirem, e ressalva comando, gate ou versão diferentes
`auth`, `pending`	ARTIFACT	A pessoa	Ação externa irreversível com a autorização citada (AUTH) e pendência que a documentação prescreve (PENDING)
`twins`	TWIN CHECK	O kata, a partir da busca	Padrão buscado e ocorrências. Auditoria: falsificado se `defect_fixed` sem `searched`
`artifact`	ARTIFACT	O kata (calculado)	Linhas devidas (`*_owed`) e presentes (`*_present`). Se AUTH é devida e falta, o JUDGE aponta ação não autorizada
`base_commit`, `approved_commit`, `judge`	JUDGE	O kata	Não aparecem no exemplo (repositório sem *commits*). Delimitam a janela de *diff* e guardam o veredito; `kata --stamp` grava o commit final em `approved_commit`

Tabela 3. Os cinco domínios do kata, parte 1: artefatos, evidência a abrir antes de agir e ordem de autoridade. A coluna de autoridade mostra a ordem que vale depois da declaração explícita do usuário, que é sempre a maior. Fonte: leitura dos adaptadores em `domains/` e do código do kata 0.8.0.

Domínio e artefatos	**Evidência a abrir antes de agir**	**Autoridade (após o usuário)**
`coding`: código de aplicação e testes	Diff, testes e especificação	Especificação, testes, código (fase INTENT)
`devops`: Docker e Compose, Terraform, Nginx, CI/CD, *deploy*, *healthchecks*	Arquivos de configuração, documentação operacional e estado do ambiente	Documentação operacional, configuração versionada, estado observado
`data-analysis`: pandas, Polars, SQL, *notebooks*, painéis, *pipelines*	Scripts, esquema dos dados e dicionário de dados	Dicionário de dados, código de análise, dados observados
`research`: APIs, bibliotecas, literatura, comparações, experimentos	Fontes primárias e código observado	Fonte primária, código observado
`docs`: README, guias, *changelog*, docstrings, especificações	Documentos vizinhos e a fonte técnica que descrevem	Fonte técnica real, convenções do repositório

Tabela 4. Os cinco domínios do kata, parte 2: verificação por observação e fraudes que cada adaptador manda o agente e o revisor procurarem. Esses tipos de fraude são instruções dos adaptadores, e não código do JUDGE. Fonte: leitura dos adaptadores em `domains/` e do código do kata 0.8.0.

Domínio	**Verificação por observação**	**Fraudes que o adaptador manda caçar**
`coding`	Lint, testes e cobertura, por `ruff` e `pytest` ou pelos comandos do projeto	As oito categorias do JUDGE (Seção 3)
`devops`	`docker compose config` e `ps`; `docker build`; `nginx -t` e *curl* no *healthcheck*; sintaxe do *workflow*	`unverified_deploy`, `weakened_healthcheck`, `secret_in_diff`, `unpinned_image`, `missing_volume_port`, `silent_config`, `unauthorized_action`, `prod_direct_edit`
`data-analysis`	Executar o script ou *notebook* com saída 0; `EXPLAIN` em amostra; conferir forma e nulos; renderizar o gráfico	`unverified_analysis`, `cherry_picking`, `data_leak`, `unpinned_dependency`, `silent_drop`, `fabricated_number`, `unauthorized_access`
`research`	Abrir o trecho da fonte; reproduzir com *script* mínimo; apontar arquivo e linha; separar observado de inferido	`fabricated_source`, `hallucinated_behavior`, `cherry_picking_sources`, `stale_source`, `unverifiable_claim`
`docs`	Rodar o comando citado; buscar a *flag* na fonte; derivar contagens da implementação	`drift_numerico`, `fabricated_command`, `stale_procedure`, `orphan_section`

Tabela 5. Leitura pedagógica das fases do kata. Os processos cognitivos (Lembrar, Compreender, Aplicar, Analisar, Avaliar e Criar) seguem a taxonomia revisada de Bloom [Anderson & Krathwohl, 2001]; a atribuição a cada fase é interpretação nossa. N/A: nenhuma verificação automática além da presença do campo. Fonte: nossa análise do código do kata 0.8.0.

Fase	Processo cognitivo	Evidência que a fase deixa	Verificação automática
FIT	Analisar	Classificação da tarefa e rota escolhida	Diff medido (arquivos e linhas); atalho para tarefas triviais
THINK	Compreender, Analisar	Problema, premissas, alternativas, incógnitas e critério de pronto	N/A
SIMPLIFY	Avaliar	Confirmação de código mínimo, sem abstração especulativa	N/A
INTENT	Analisar, Avaliar	O que o código faz, o que o teste espera e o que a especificação diz	Desacordo não resolvido é fraude de alta gravidade no JUDGE
SURGICAL	Avaliar	Arquivos alterados, cada um justificado	Comparação com o diff real (escopo excedente)
VERIFY	Aplicar, Avaliar	Resultados de lint, testes e cobertura	Comandos executados e códigos de saída
TWIN CHECK	Aplicar, Analisar	Padrão do defeito e ocorrências encontradas	Busca textual no projeto
ARTIFACT, REPORT	Avaliar, Criar	Linhas devidas e relatório com o resultado primeiro	Presença das linhas devidas
JUDGE	Avaliar (autocrítica)	Veredito e fraudes encontradas	Reexecução e comparação com o Git

Tabela 6. Do risco nomeado pela auditoria a uma pergunta formativa. A coluna central reproduz as mensagens do kata 0.8.0 (`_AUDIT_RISKS`, `src/kata/report.py`), traduzidas para o texto corrido e com "spec" escrito por extenso; a coluna da direita é sugestão nossa, não validada.

Fase	Risco que o kata nomeia	Pergunta formativa sugerida
FIT	rota e trivialidade não classificadas por humano: esforço pode ser desperdiçado em tarefa trivial ou mal roteada	Por que esta tarefa é, ou não é, trivial? O que mudaria sua escolha de rota?
THINK	*assumptions* nunca declaradas: qualquer solução pode atacar o problema errado	Qual premissa, se falsa, derrubaria sua solução? Como você a testaria?
SIMPLIFY	minimalidade afirmada sem ninguém confrontar o *diff* com o pedido: abstrações especulativas podem passar sem revisão	Que parte da mudança você removeria sem deixar de atender ao pedido?
SURGICAL	cada arquivo declarado necessário sem ninguém conferir: escopo extra pode entrar sem ser notado	Qual arquivo você tocou que o pedido não exige?
INTENT	código, teste e especificação podem discordar sem registro: comportamento muda sem intenção verificada	Quando código e teste discordam, quem tem razão, e por quê?
VERIFY	sucesso afirmado sem evidência de execução: a tarefa pode estar aprovada sobre nada	O que a saída deste comando prova, e o que ela não prova?
TWINS	defeito corrigido sem busca de recorrência: o mesmo padrão pode se repetir em outros lugares	Onde mais este mesmo erro poderia estar?

Tabela 7. Casos de uso propostos para o kata na educação em software. Os casos 1 e 2 são os do primeiro teste e do protocolo, e os casos 3 a 8 são extensões. Todos são hipóteses de projeto. Fonte: elaboração nossa a partir dos recursos do kata 0.8.0.

Nº	Caso de uso	Comandos principais	Evidência para o docente	Principal ressalva
1	Planejar antes de pedir código ao agente	`--plan`; `--check-only`	Registro de problema, premissas e critério de pronto	Só se verifica que o campo foi preenchido
2	Ensino de testes e desenvolvimento guiado por testes	`--judge`	Fraude de teste enfraquecido, com o arquivo apontado	Detecta padrões sintáticos, não a qualidade do teste
3	Integridade em entregas com agentes de IA	`--check-only`; `--judge --trusted-base`	Veredito e fraudes na integração contínua	Desvios do Git; veredito é indício, não prova
4	Correção de defeitos em repositórios reais	ciclo completo; TWIN CHECK	Arquivo de tarefa e busca do mesmo padrão	Tarefas em Python; custo de tempo
5	Projeto integrador em equipe	`--audit`; `--report`	Notas por fase e relatório por contribuição	Conteúdo plausível e falso é graduado como seguido
6	Especificação, testes e intenção	fase INTENT; `--judge`	Registro de conflito e de sua resolução	Resolução é autodeclarada
7	Retirada gradual do andaime	rotas do FIT; política da disciplina	Quais fases o estudante registrou	Sem comando para dispensar fases; sem evidência de que a retirada ajuda
8	Outras linguagens e outros artefatos	`.kata/config.yaml`; adaptadores de domínio	Mesmo registro, com os comandos do projeto	Verificação técnica só em Python; adaptadores não avaliados

Tabela 8. Tamanho de amostra por grupo em cada hipótese (poder 0,80). Os efeitos são suposições de planejamento. A perda considerada é de 15%. Fonte: nosso cálculo por aproximação normal; para proporções, variância combinada sob a hipótese nula no numerador (com variâncias separadas, a H2 pede 118 sem perda; com correção de continuidade, cerca de 133).

Hipótese e efeito suposto	**Nível**	**Sem perda**	**Com perda**
H2 (principal): 30% contra 15% de entregas com falha oculta	0,05	121	142
H1: *d* de Cohen de 0,5	0,0167	84	99
H3: não inferioridade, margem de 0,5 DP	0,0167 (unilateral)	71	84
H4: *d* de Cohen de 0,8	0,0167	33	39

Tabela 9. O que cada execução real mostra. Todas foram feitas por nós em projetos descartáveis; as saídas completas estão em `demos/saidas/`. Fonte: execução do kata 0.8.0 por `demos/reproduzir.sh`.

Nº	Domínio	Comando	Resultado (saída)	O que mostra
1	`coding`	`--init`, `--task`, `--audit`	Aprovado; auditoria limpa (0)	O ciclo completo e o registro por fase, com a resolução do conflito em branco
2	`coding`	`--judge` (duas vezes)	`REFUTED` (1), depois `VERIFIED` (0)	O JUDGE cobra a resolução do conflito, mas aceita qualquer frase sem verificá-la
3	`coding`	`--judge`, `--check-only`	`REFUTED` (1); rejeitado (1)	Testes e cobertura passam com o teste vazio; quem acusa é o JUDGE, e o lint só por efeito colateral
4	`coding`	`--audit`	7 fases *faked* (1)	A auditoria acusa campo vazio, e não conteúdo plausível e falso
5	`docs`	`--check-only` (duas vezes)	Rejeitado (1), depois aprovado (0)	Verificação fora de Python, declarada pelo projeto, com a cobertura de documentação como métrica
6	`data-analysis`	`--check-only` (duas vezes)	Rejeitado (1), depois aprovado (0)	Fraude de remoção silenciosa de linhas, que o adaptador só manda vigiar, vira verificação executável declarada pelo projeto
7	`devops`	`--check-only` (duas vezes)	Rejeitado (1), depois aprovado (0)	Imagem sem versão fixa e *healthcheck* desativado, que o adaptador só manda vigiar, viram verificação executável do projeto
8	`research`	`--check-only` (duas vezes)	Rejeitado (1), depois aprovado (0)	Fonte inexistente e afirmação sem fonte são pegas por um *script* do projeto, com rede